Port City International University
Department of Computer Science and Engineering

Lettuce Leaf Disease Classification using Deep Learning

Supervisor
Mr. Shafayet Nur
Lecturer, Department of CSE, PCIU
shafayetnur217@gmail.com

Author
Mohammad Zunaed Kawser
ID: CSE 02507319
zunayedkawser79@gmail.com

This thesis is submitted to the Department of Computer Science and Engineering of Port City International University in fulfillment of the requirements for the degree of Bachelor of Science

19 June, 2025

Declaration

It's hereby declared that this thesis has not been submitted elsewhere for the award of any degree to the best knowledge of the author's references and acknowledgements to other researchers have been given as appropriate. I also confirm that I have just used the resources that have been indicated. All formulations and concepts borrowed directly or in substance from printed or unprinted material, or the Internet, have been cited according to proper scientific procedure, with footnotes or other precise references to the original source.

I am aware that providing inaccurate information may result in legal ramifications.

(Signature of the Candidate)

Mohammad Zunaed Kawser
ID: CSE 02507319,
Department of Computer Science
and Engineering
Port City International University

Approval for Submission

This thesis, titled "**Lettuce Leaf Disease Classification using Deep Learning,**" was submitted to the Department of Computer Science and Engineering, Port City International University, in partial fulfillment of the requirements for the degree of Bachelor of Science (Engineering).

(Signature of the supervisor)

Mr. Shafayet Nur
Lecturer Department of
Computer Science and Engineering
Port City International University
7, Nikunja Housing Society, South Khushi,
Chattogram- 4202.Chattogram, Bangladesh.
Email: shafayetnur217@gmail.com
Telephone: +8801933648557

Dedication

This thesis is dedicated to all farmers, hardworking people, my parents and honorable teachers.

Acknowledgements

First and foremost, offer my sincere thanks to almighty Allah for giving to me the heavenly gift of effectively finishing my final year thesis. I had the astounding honor of concentrating on under the direction and consolation of my regarded supervisor, Mr. Shafayet Nur Sir. His tremendous information offers me the chance to expand my viewpoints and gain significant headway.

Abstract

The increasing global demand for food requires advanced solutions for crop health monitoring to ensure sustainable agriculture. This research focuses on leveraging deep learning techniques to automatically detect and classify the health status of lettuce plants using image data. Two publicly available datasets were collected and balanced using data augmentation, encompassing six categories: alive, dead, growing, raising seeding, sclerotinia rot, and downy mildew. A series of convolutional neural networks (CNNs), including Xception, DenseNet121, EfficientNetB0, and ConvNeXtTiny, along with transformer-based models like ViT, were evaluated. The study introduced a hybrid model combining ConvNeXtTiny and EfficientNetB0, achieving a classification accuracy of 97%. The model's performance was assessed using accuracy, precision, recall, F1-score, and confusion matrices. This work demonstrates the potential of hybrid deep learning models in real-time crop disease detection and provides a robust framework for precision farming systems.

Contents

Declaration i

Approval for Submission ii

Dedication iii

Acknowledgement iv

Abstract v

1 Introduction 13

1.1 The Role of AI in Crop Health Monitoring---------- 14

1.1.1 Computer Vision ---------- 14

1.1.2 Deep Learning ----------

1.1.3 Convolutional Neural Networks (CNNs) ----------

1.1.4 The Private Mobile Network (PMN) Backbone) ----------

1.2 Problem Definition ----------

1.3 Objectives ----------

2 Literature Review

3 Methodology

3.1 Data Collection ----------

3.2 Preprocessing ----------

3.3 Data Labeling ----------

3.4 Splitting into Train and Test Data ----------

3.5 Deep Learning Algorithm ----------

3.6.1 Convolutional Neural Network----------

3.6 Transformer-Based Algorithm ----------

3.6.1 ViT----------

3.6.2 ViT vs CNN----------

3.7 Comparative Performance Analysis ----------

4 Hardware Implementation and Toolkit

4.1 Tools and Devices ----------

4.1.1 Python----------

4.1.2 Colaboratory Notebook----------

4.1.3 Google Scholar ----------

4.1.4 kaggle----------

4.1.5 Roboflow----------

4.2 Hardware Implementation ----------

5 Result and Analysis

5.1 Confusion Matrix----------

5.1.1 Confusion Matrix of all CNN algorithms ----------

5.1.2 Confusion Matrix of the Transformer algorithm ----

5.2 Performance Curves----------
5.2.1 Performance Curves of all CNN algorithms ----------
5.2.2 Performance Curves of Transformer algorithm -------
5.3 Precision, Recall and F1-Score ----------
5.4 Average accuracy score of all algorithms for all models--------
5.5 Comparison with Previous Work ----------
5.6 Snaps of The Final Models ----------

6 Future Works & Conclusion

6.1 Future Work ----------
6.2 Limitation ----------
6.3 Conclusion ----------
6.4 Reference----------

7 Appendix

7.1 Preprocessing ----------
7.1.1 Augmentation----------
7.2 Xceptioin Model ----------
7.3 Hybrid Model (ConvNextT+EfficientNetB0) ----------

List of Figures

Figure 3.1: Proposed Methodology

Figure 3.2 Visualization of dataset 1

Figure 3.3 Visualization of dataset 2

Figure 3.4: Data distribution of the dataset

Figure 3.5: Layer combination Input of the CNN architecture

Figure 5.1: Confusion Matrix of Deep Learning algorithms

Figure 5.1: Confusion Matrix of the Transformer-based algorithm

Figure 5.1: Performance Curves of Deep Learning algorithms

Figure 5.1: Performance Curves of the Transformer-based algorithm

Figure 5.4: Accuracy for the dataset

Figure 5.6: Curve by ConvNextT + EfficientNetB Model

Figure 5.6: Result of ConvNextT + EfficientNetB Model

Figure 5.6: ROC Curve of ConvNextT + EfficientNetB Model

List of Tables

Table 3.3: Shows data distribution of all 6 classes.

Table 3.4: Train and test data size

Table 3.6: Difference between CNN and ViT

Table 5.3: Detailed score of Deep Learning algorithms for the dataset

Table 5.5: Comparison of two approaches

Chapter 1

Introduction

In the modern era, technology is the backbone of innovation across all sectors, from global communication to critical industries like agriculture. As the world's population continues its upward trend, projected to reach nearly 10 billion by 2050, the demand for a stable and secure food supply has never been more pressing. A key factor in ensuring food security is protecting crop yields from diseases and stress, which can devastate harvests and lead to significant economic losses. Traditional methods of monitoring crop health rely on manual inspection, a process that is often slow, labor-intensive, and prone to human error, allowing diseases to spread before they are accurately identified.

This global challenge necessitates a paradigm shift towards more intelligent and proactive farming practices. Precision Agriculture emerges as a transformative solution, applying data-driven techniques to optimize farming operations. Specifically for crop health, this means moving from reactive treatment to proactive, real-time monitoring. By observing and responding to the earliest signs of disease or stress in individual plants, farmers can intervene quickly and effectively.

The engine driving this agricultural revolution is the convergence of Artificial Intelligence (AI), high-resolution imaging, and next-generation connectivity like 5G. AI, particularly through deep learning, provides the diagnostic "brains" for this new generation of farming systems, enabling them to learn, predict, and identify problems with superhuman accuracy.

1.1 The Role of AI in Crop Health Monitoring

Artificial Intelligence is fundamentally reshaping how we protect our crops. Instead of relying solely on visual checks, farmers can now deploy intelligent systems to diagnose plant health with unprecedented accuracy and scale. Two key areas of AI are particularly impactful:

1.1.1 Computer Vision

Computer Vision is a field of AI that trains computers to interpret and understand the visual world. Using digital images from cameras in the field, this technology can see and analyze subtle signs of distress in crops. In crop health monitoring, this is used for:

- **Early Disease Detection:** Identifying microscopic signs of fungal or bacterial diseases like *sclerotinia rot* or *downy mildew* on plant leaves before they are obvious to the naked eye.
- **Nutrient Deficiency Analysis:** Detecting discoloration or patterns that indicate a lack of essential nutrients.
- **Pest Damage Identification:** Recognizing the specific damage patterns left by harmful insects.

1.1.2 Deep Learning

Deep learning is a part of AI that learns information through artificial neural networks, which are calculations inspired by the human brain. The deep learning calculation would rehash a task, modifying it slightly each time to improve the outcomes, much like how people learn through experience.

Neural networks are layers of nodes, similar as the human cerebrum is comprised of neurons. Nodes inside individual layers are associated with adjoining layers. The network is supposed to be deeper in view of the number of layers it has. A solitary

neuron in the human mind gets a great many signals from different neurons. In an artificial neural network, signals travel among nodes and distribute comparing load. A heavier-weighted node will have more impact on the following layer of nodes. The last layer orders the weighted contributions to deliver a result. Deep learning frameworks require robust equipment, as they handle a large amount of information and involve complex numerical computations.

1.1.3 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) are the gold standard for image analysis. They are designed to automatically learn the most important features from images, making them ideal for telling the difference between a healthy leaf and a diseased one. This project utilizes the Xception model, a powerful and efficient CNN architecture, for the specific task of classifying the health status of lettuce crops based on image data.

1.1.4 The Private Mobile Network (PMN) Backbone

A truly effective smart-farming system requires the rapid transmission of high-quality data. High-resolution images, which are essential for accurate AI-based diagnosis, are large files that can be difficult to transmit from a remote field over standard networks. Any delay between capturing an image and getting a diagnosis can allow a disease to spread.

This research proposes the use of a 5G Private Mobile Network (PMN) as the data backbone. A 5G PMN offers a dedicated, high-bandwidth, and low-latency communication network. This ensures:

- **Real-time Diagnostics:** Seamless streaming of high-resolution images from field cameras to the AI model for immediate analysis.
- **Reliable Data Flow:** A stable connection that is not affected by public network congestion, crucial for consistent monitoring.
- **Enhanced Security:** A private network that protects sensitive farm data, a feature further enhanced in the broader system concept with Quantum Key Distribution Function (QKDF) integration.

1.2 Problem Definition

The core problem this research addresses is the critical need for fast, accurate, and scalable crop disease detection. The limitations of manual inspection can lead to delayed responses, resulting in reduced crop yield, lower quality produce, and increased reliance on broad-spectrum chemical treatments. Farmers require an automated diagnostic tool that can:

1. Accurately identify various health statuses in crops, from healthy to different types of disease.
2. Operate in near real-time to provide timely alerts.
3. Leverage a secure and reliable communication network to function effectively in a farm environment.

This project is directed to mitigate these gaps by developing a specialized deep learning model focused specifically on classifying the health status of lettuce, thereby providing a powerful tool for modern precision agriculture.

1.3 Objectives

The main objectives of this research are as follows:

- To build and optimize a specialized deep learning model based on the Xception architecture for the primary purpose of classifying crop health status.
- To train the model on a labeled dataset of lettuce images, categorizing them into classes such as 'alive', 'sclerotinia_rot', and 'downy_mildew'
- To build an efficient and robust model that achieves high diagnostic accuracy (benchmarked at 87%) for reliable disease detection.
- To design the system to operate over a 5G PMN, ensuring the efficient data transmission required for real-time health monitoring.

Chapter 2

Literature Review

A literature review is a comprehensive summary and analysis of previous research on a particular topic. It helps identify what has already been studied, the methods used, findings obtained, and gaps that remain. In research projects, the literature review lays the foundation for understanding the current state of knowledge. It guides the direction for new research by showcasing the relevance and contribution of existing works.

The first paper, "A Novel 5G PMN-Driven Approach for AI-Powered Irrigation and Crop Health Monitoring," was authored by Tang Nguyen-Tan and Quan Le-Trung and published in IEEE Access (2024). This study focuses on applying AI techniques in agricultural irrigation and health monitoring of lettuce crops. It integrates a variety of datasets, including the Lettuce Growth Stage Detection Dataset, Okelafe 2 Dataset, Nutritional Deficiency Dataset, and a self-collected dataset from lettuce fields in Da Lat, Vietnam.

The researchers utilized a Lightweight Transformer-Based Model along with YOLOv8n for model development. For irrigation, the Irrigation Scheduling Model achieved 73% accuracy, supporting water-efficient decision-making. The YOLOv8 model gave strong performance across three parameters: Growth Stage Detection with 93.8% accuracy, Health Status Classification with 87% accuracy, and Nutritional Status Detection with 83% accuracy. This study stands out for its integration of real-time communication (5G PMN), diverse datasets, and high-performance lightweight AI models.

The second paper, “Implementation of Artificial Intelligence in Agriculture for Optimisation of Irrigation and Application of Pesticides and Herbicides,” was authored by Tanha Talaviya, Dhara Shah, Nivedita Patel, Hiteshri Yagnik, and Manan Shah, and published by Elsevier B.V. on behalf of KeAi Communications Co. Ltd in 2020. Unlike the first study, this paper does not rely on a specific dataset but instead reviews broad AI applications in agriculture.

The authors propose neural moderation models and machine vision systems for improving irrigation efficiency and agrochemical application. The study presents a generalized approach, showing that such models can achieve up to 92% accuracy in real-world scenarios. This literature contributes a more theoretical perspective by highlighting the potential of AI across different agricultural domains rather than focusing on one dataset or crop.

The third paper, “Fabrication and Investigation of Agricultural Monitoring System with IoT & AI,” was published in SN Applied Sciences in 2023 and authored by P. Indira, I. Sheik Arafat, R. Karthikeyan, Shitharth Selvarajan, and Praveen Kumar Balachandran. This research focuses on integrating IoT sensors and AI models for smart agricultural monitoring.

The study used NDVI (Normalized Difference Vegetation Index) time series analysis to assess crop health and growth. For model implementation, Keras-RetinaNet and ResNet50 backbone were used. The approach achieved 79% accuracy, offering an effective balance between hardware-based sensing and AI-driven analytics. This work demonstrates the practicality of combining image-based data with IoT for real-time and scalable monitoring systems.

In my research work, I focused on the first paper, specifically utilizing the Okelafe 2 Dataset and concentrating on the Health Status Classification task, which achieved an accuracy of 87%.

Chapter 3

Methodology

This chapter provides an extensive outline of the research technique. Every one of the information assortments, preprocessing, word embedding, framework design, and performance analysis matrices are made shown in this section.

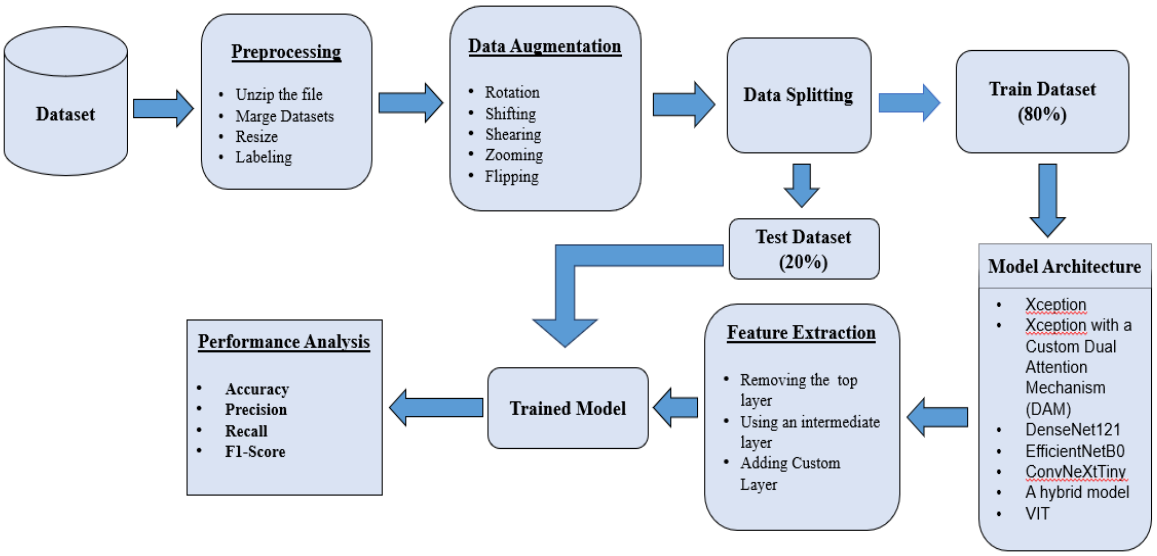

Figure 3.1: Methodology

3.1 Data Collection

This research combines two datasets. Both datasets are publicly available. The first dataset has only two classes, and the other one has four classes. It includes 18372 labeled data. These data are available on Roboflow. (Roboflow is a web-based platform used for managing datasets and training computer vision models)

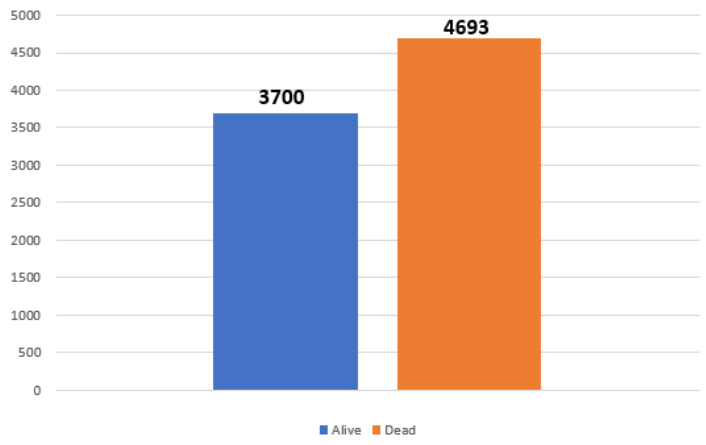

Figure 3.2: Visualization of dataset 1

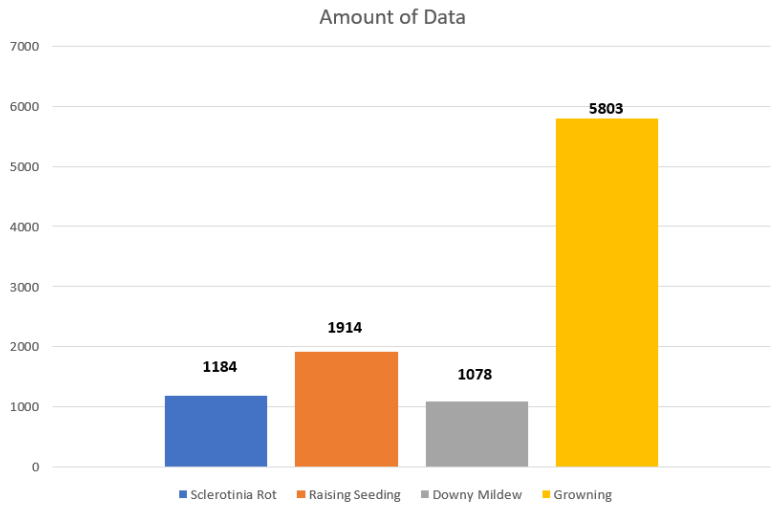

Figure 3.3: Visualization of dataset 2

3.2 Preprocessing

Effective preprocessing is essential for achieving high performance in deep learning models, especially when working with real-world image datasets. Our dataset initially contained noise, inconsistencies, and imbalanced class distributions, all of which could negatively affect training. To address these issues, we applied the following preprocessing steps:

- **Unzipping the Dataset:** The first step involved extracting image files from the Merge Lettuce Dataset.zip. These images were then organized into folders based on their respective health status classes (e.g., alive, dead, growing).

- **Resizing Images:** All images were resized to a uniform size of 128×128 pixels using cv2.resize(). This step ensures consistency in input dimensions, which is a requirement for feeding data into convolutional neural networks.

- **Filtering Unreadable Files:** While loading images using cv2.imread(), we included a check (if image is not None) to automatically skip corrupted, unreadable, or unsupported image files. This prevented errors during training and ensured the dataset only contained valid images.

- **Label Cleaning and Consistency:** During the manual labeling phase, we reviewed the dataset to correct mislabels and remove irrelevant or misleading samples. This helped improve label accuracy and ensured that each image correctly represented its class.

- **Class Balancing Using Data Augmentation:** The original dataset had an unequal number of images across classes, which can cause model bias. To address this, we set a target of 5,800 images per class. For classes with fewer samples, we applied data augmentation techniques such as:

    - Rotation
    - Shifting
    - Shearing
    - Zooming
    - Horizontal Flipping

These transformations helped generate diverse training data while preserving the integrity of each class, resulting in a balanced and more generalizable dataset.

3.3 Data Labeling

After collecting the data from Roboflow, all the data are manually labelled as alive, dead, growing, sclerotinia rot, raising seeding, and downy mildew.

Class	Amount of Data
Alive	3700
Dead	4693
Sclerotinia Rot	1184
Raising Seeding	1914
Downy Mildew	1078
Growing	5803
Total	18372

Table 3.3: shows the data distribution of all 6 classes.

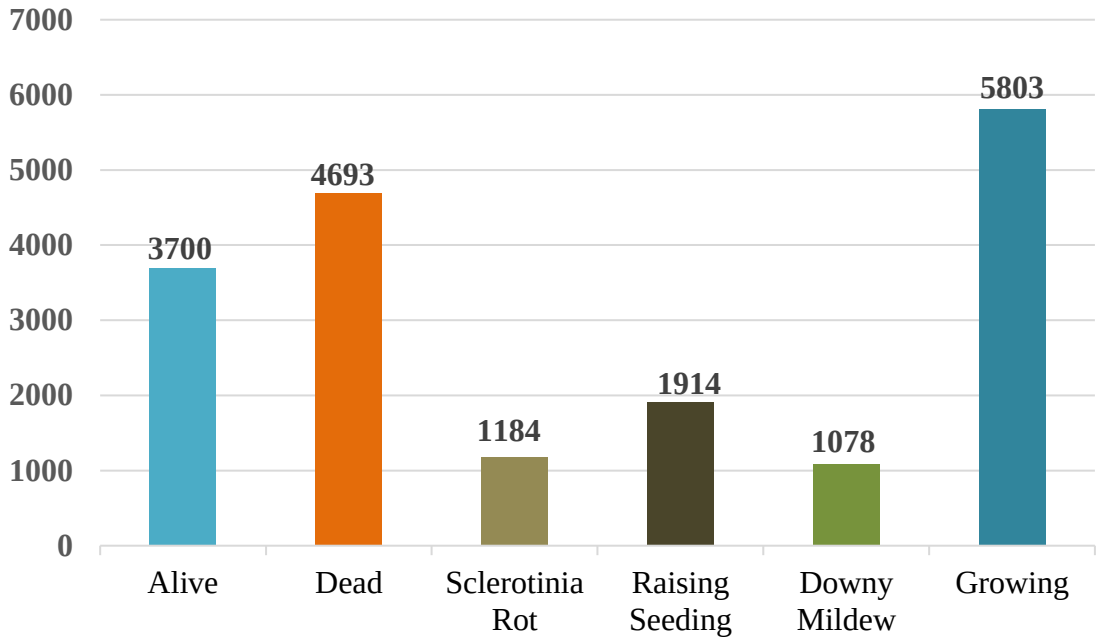

Figure 3.4: Data distribution of the dataset

To balance the imbalanced dataset, each class was upsampled to a target count of 5,800 images using data augmentation. Original images were resized to 128×128, and the shortfall in minority classes was filled using ImageDataGenerator, applying techniques like rotation, shift, shear, zoom, and flip. Only underrepresented classes were augmented, while the majority classes were left unchanged. This ensured class balance through upsampling. The final dataset contained 34,803 images.

3.4 Splitting into Train and Test Data

- **Train-Test Split:** The code uses (sklearn.model_selection.train_test_split) to divide the data into training and testing sets.

- **Split Ratio:** The (test_size = 0.2) parameter indicates that 20% of the data is allocated to the testing set, and the remaining 80% is used for training.

- **Shuffling:** Before splitting, the data (images and labels) is shuffled using (sklearn.utils.shuffle) with a random state. Shuffling ensures that the training and testing sets are representative of the overall dataset and prevents any ordering bias.

- **Purpose:** As stated in the code, the test set provides an unbiased evaluation of the final model [1]. Splitting the data into training and testing sets is essential to assess how well the trained model generalizes to unseen data and to detect overfitting.

Training data	27,842 samples (80.00%)
Testing data	6,961 samples (20.00%)
Total data	34803 samples

Table 3.4: Train and Test data size

We initially used a 70/30 split, but it did not perform well. Training accuracy was lower, and the model showed signs of underfitting due to limited training data. Switching to an 80/20 split improved performance by giving the model more data (80%) to learn patterns effectively while still reserving enough (20%) for reliable testing. This balance is especially important in deep learning, where larger training sets often lead to better generalization and accuracy.

3.5 Deep Learning Algorithm

This part is for introducing the Deep learning models that were used in this research.

3.5.1 Convolutional Neural Network

Xception, DenseNet121, EfficientNetB0, and ConvNeXtTiny are all types of Convolutional Neural Networks. CNNs are a class of deep learning models specifically designed for processing data that has a grid-like topology, such as images. They use convolutional layers to automatically learn hierarchical features from the input data.

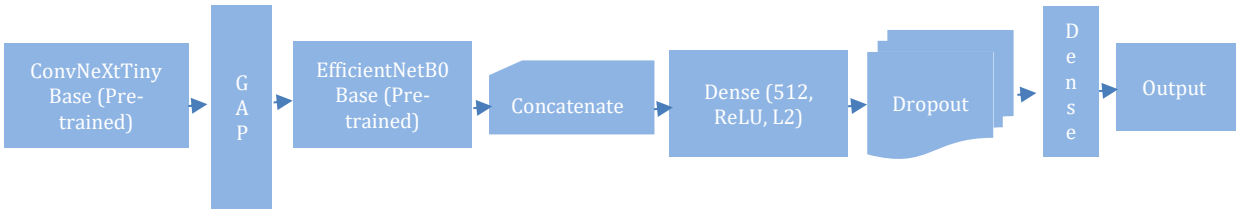

Figure 3.5: Layer combination Input of the CNN architecture

Input Layer

- Receives the raw image data (e.g., a 3D tensor representing height, width, and color channels).

Dropout Layer

- Used during training to randomly drop out a fraction of the neurons in a layer.
- This prevents the network from becoming too reliant on specific neurons and helps to reduce overfitting.
- Often placed after dense layers.

Convolutional Layer (Conv2D)

- These are the core building blocks of a CNN.
- They apply filters (kernels) to the input image to extract features.
- Each filter detects a specific pattern or feature (like edges, corners, textures).
- The output of a convolutional layer is a set of feature maps.
- Multiple convolutional layers are often stacked, with later layers learning more complex and abstract features based on the features learned by earlier layers.
- Combination with Activation*:* Convolutional layers are usually followed by a non-linear activation function (like ReLU) to introduce non-linearity into the model, allowing it to learn more complex relationships.

Pooling Layer (MaxPooling2D, AveragePooling2D):

- These layers reduce the spatial dimensions (height and width) of the feature maps.
- They help to reduce the number of parameters and computation in the network, making it more efficient.
- Pooling also helps to make the learned features more robust to small variations in the position of features in the input image.
- MaxPooling takes the maximum value within a defined window, while AveragePooling takes the average.

Flatten Layer

- Used to convert the multi-dimensional output of the convolutional and pooling layers into a 1D vector.
- This is necessary before feeding the features into fully connected layers.

Dense Layer

- These are standard neural network layers where every neuron in the layer is connected to every neuron in the previous layer.
- They take the flattened features and learn to combine them for the final classification.
- Multiple dense layers can be used.
- Combination with Activation: Dense layers are also typically followed by activation functions (like ReLU).

Batch Normalization Layer

- Often inserted between convolutional/dense layers and activation functions.
- Helps to stabilize the learning process and allows for the use of higher learning rates.
- Normalizes the activations of the previous layer.

Activation Functions (e.g., ReLU, Sigmoid, Tanh):

- Applied after convolutional layers to introduce non-linearity.
- ReLU (Rectified Linear Unit) is a very common choice due to its computational efficiency.

Output Layer

- The final dense layer that outputs the predicted class probabilities.
- The number of units in this layer is equal to the number of classes in the classification problem.
- For multi-class classification (like yours with 6 classes), the softmax activation function is used to produce a probability distribution over the classes. For binary classification, a sigmoid activation is used.

Several established deep learning architectures and custom hybrid models are explored for this classification task. The models investigated include:

Xception:

Xception (Extreme Inception) is a deep convolutional neural network architecture that relies entirely on depthwise separable convolutions. This design makes it highly efficient in terms of computational resources while still achieving strong performance. In this research, the Xception model achieved an accuracy of 96%, demonstrating its capability in capturing both spatial and hierarchical features from complex leaf images. Its lightweight structure and high accuracy make it well-suited for real-time plant disease classification tasks, especially on moderately powerful hardware.

Xception + DAM (Dual Attention Mechanism):

This enhanced model builds upon the standard Xception architecture by integrating a Dual Attention Mechanism, consisting of both Position and Channel Attention modules. The Position module allows the model to focus on spatial regions of interest, while the Channel module helps it prioritize the most significant feature maps. This attention-based enhancement led to a 96% accuracy, improving the model's sensitivity to subtle leaf abnormalities. It is beneficial in scenarios where disease symptoms are fine-grained and localized.

DenseNet121:

DenseNet121 (Densely Connected Convolutional Network) is designed to maximize feature reuse by connecting each layer directly to every other layer in a feed-forward fashion. This dense connectivity promotes improved gradient flow, efficient parameter usage, and high feature diversity. Achieving 92% accuracy in this study, DenseNet121 proved highly effective in recognizing complex visual cues despite a relatively compact architecture. It is ideal for applications with limited

computational resources or training data, where model efficiency and generalization are key.

EfficientNetB0:

EfficientNetB0 is part of a family of models that use compound scaling to balance network depth, width, and resolution. It is known for delivering strong performance with fewer parameters, and in this study, it reached an accuracy of 94%. Its efficient design enables fast inference and lower memory usage without sacrificing predictive power. This makes it a strong candidate for deployment on mobile devices or edge computing systems in agricultural monitoring.

ConvNeXtTiny:

ConvNeXtTiny is a modernized convolutional neural network architecture inspired by Vision Transformers. It incorporates large kernel sizes, Layer Normalization, and GELU activation, which help it achieve high accuracy while maintaining the inductive biases and efficiency of traditional CNNs. With a performance of 94% accuracy, it offers transformer-level representation power in a CNN framework. Its design is particularly well-suited for precision agriculture systems that demand a blend of performance and scalability.

ConvNeXtTiny + EfficientNetB0:

This hybrid model combines the strengths of ConvNeXtTiny and EfficientNetB0 by concatenating their extracted features before classification. ConvNeXtTiny captures fine-grained spatial details using its modern CNN structure, while EfficientNetB0 contributes with its efficient compound scaling and strong contextual awareness. Together, they form a powerful classifier that achieved the highest accuracy of 97%, offering a robust balance between depth, efficiency, and performance in multi-class crop disease detection.

ConvNeXtTiny + DenseNet121:

By fusing ConvNeXtTiny's modern convolutional features with DenseNet121's densely connected pathways, this hybrid model benefits from both deep hierarchical feature learning and strong gradient flow. DenseNet121 contributes to efficient parameter usage and feature reuse, while ConvNeXtTiny adds spatial precision. The combination leads to high performance (96% accuracy) in detecting subtle differences between similar plant disease classes.

Xception + ConvNeXtTiny:

This model integrates Xception's depthwise separable convolution efficiency with ConvNeXtTiny's transformer-inspired convolutional design. Xception ensures low computational overhead while capturing hierarchical features, and ConvNeXtTiny enhances spatial representation. The resulting fusion offers a lightweight yet highly effective model with 96% accuracy, suitable for deployment on devices with moderate resource constraints.

DenseNet121 + EfficientNetB0:

This hybrid model merges DenseNet121's deep feature connections with EfficientNetB0's scalable design. DenseNet121 ensures feature reuse and improved learning depth, while EfficientNetB0 contributes efficient and balanced learning across depth, width, and resolution. Though slightly lower in performance (93% accuracy), this model is particularly effective in scenarios demanding strong generalization with limited computational capacity.

Xception + DenseNet121 + EfficientNetB0:

A triple-fusion model that combines three powerful CNNs to extract a wide range of features—from localized patterns to deep abstract representations. Xception handles efficiency, DenseNet121 adds dense connectivity, and EfficientNetB0 contributes balanced scaling. Despite its architectural complexity, the model

maintains 92% accuracy and provides a diverse feature space that benefits exploratory research and model interpretability.

ConvNeXtTiny + EfficientNetB0 + DenseNet121:

This ensemble integrates ConvNeXtTiny's modern design, EfficientNetB0's compound scalability, and DenseNet121's dense connections. The triple fusion enables the model to capture spatial, contextual, and hierarchical features simultaneously, resulting in 96% accuracy and outstanding F1-scores. It is particularly strong at distinguishing between closely related health statuses such as 'growing' and 'alive', making it ideal for real-world agricultural applications where precision is critical.

Data augmentation is applied using Image Data Generator with various transformations such as rotation, shifting, shearing, zooming, and horizontal flipping to increase the diversity of the training data and improve model generalization. The models are trained using the Adam optimizer and the categorical cross-entropy loss function. Performance is evaluated using metrics including accuracy, precision, recall, F1 score, and loss. Classification reports and confusion matrices are used to analyze the performance of each model across the different classes. ROC curves and AUC values are also presented to assess the models' ability to distinguish between classes.

3.6 Transformer Algorithm

The Transformer is a deep learning model originally developed for natural language processing (NLP). Its core strength is the self-attention mechanism, which allows it to capture relationships across the entire input. In computer vision, this architecture is adapted as the Vision Transformer (ViT), offering a new approach to image classification by processing images as sequences of patches, unlike traditional CNNs.

Step-by-Step Workflow of Vision Transformer (ViT):

- Image Patching: The input image is divided into a grid of fixed-size, non-overlapping patches (e.g., 16×16 pixels). This is analogous to breaking a sentence into words.
- Flattening Patches: Each patch is flattened into a 1D vector to convert the spatial data into a sequential format suitable for processing by the transformer.
- Linear Projection (Embeddings): These flattened vectors are passed through a linear projection (fully connected layer) to produce lower-dimensional patch embeddings. This transforms raw pixel values into learned vector representations.
- Positional Embedding: Because the Transformer architecture does not inherently understand position or spatial relationships, positional embeddings are added to each patch embedding. This helps the model retain the original spatial layout of the image.
    - Transformer Encoder: The sequence of embedded patches is input to a standard Transformer encoder, which consists of multiple identical layers. Each layer contains:
    - Multi-Head Self-Attention: Learns contextual relationships between all patches simultaneously.
    - Feed-Forward Network: Applies non-linear transformations to enhance learned features.
    - Layer Normalization & Residual Connections: Improve training stability and model depth.
- Pretraining and Fine-tuning: The model is typically pretrained on a large labeled dataset, allowing it to learn rich visual representations. It can then be fine-tuned for specific tasks, such as plant disease classification.
- Classification Head: After encoding, the output corresponding to a special [CLS] token (or averaged patch outputs) is passed through a fully connected (dense) layer with softmax, which predicts the final class label.

Vision Transformers (ViTs) offer key advantages over traditional CNNs by leveraging global attention, which allows them to capture long-range dependencies across different parts of an image. Unlike CNNs, they are not constrained by local receptive fields, making their architecture more flexible. With enough training data and computational resources, ViTs often surpass CNNs in large-scale image tasks. In this research, the Vision

Transformer achieved 93% accuracy, showing strong capability in classifying complex lettuce leaf health conditions with good generalization.

Here are some comparisons between CNN and ViT:

Feature	**CNN (Convolutional Neural Network)**	**ViT (Vision Transformer)**
Input Handling	Processes image using local filters (convolutions)	Splits image into patches and treats as a sequence
Focus	Captures local features (edges, textures) first	Captures global relationships directly
Architecture	Hierarchical feature extraction (layer by layer)	Pure Transformer encoder with self-attention
Inductive Bias	Strong built-in spatial bias	Minimal bias – learns from data
Performance	Works well on smaller datasets	Needs more data to perform well

Table 3.6: Difference between CNN and ViT

3.7 Comparative Performance Analysis

In this exploration, the relative exhibition examination is introduced concerning the disarray matrix, exactness, precision, recall, and F1-score.

Accuracy

Accuracy is the ratio of correctly predicted instances to the total number of instances.

$$\text{Accuracy} = \frac{(TP + TN)}{(TP + TN + FP + FN)}$$

Precision

Precision is the ratio of correctly predicted positive instances to the total number of instances predicted as positive.

$$\text{Precision} = \frac{TP}{(TP + FP)}$$

Recall

Recall (also known as Sensitivity or True Positive Rate) is the ratio of correctly predicted positive instances to the total number of actual positive instances.

$$\text{Recall} = \frac{TP}{(TP + FN)}$$

F1-Score

The F1-Score is the harmonic mean of Precision and Recall. It provides a balance between these two metrics.

$$\text{F1-Score} = \frac{2 * (\text{Precision} * \text{Recall})}{(\text{Precision} + \text{Recall})}$$

Where:

- **TP (True Positive):** Instances that are actually positive and are predicted as positive.
- **TN (True Negative):** Instances that are actually negative and are predicted as negative.
- **FP (False Positive):** Instances that are actually negative but are predicted as positive (Type I error).
- **FN (False Negative):** Instances that are actually positive but are predicted as negative (Type II error).

Chapter 4

Hardware Implementation and Toolkit

4.1 Tools and Devices

The accompanying tools will be utilized in the execution of the designed system:

- Python
- Google Colaboratory (Colab)
- Google Scholar
- Kaggle
- Roboflow

4.1.1 Python

Python is a well-known, universally useful programming language that can be utilized for a wide range of applications. Python is intended to be exceptionally lucid. It utilizes English catchphrases as often as possible, whereas different dialects use accentuation, and it has fewer grammatical developments than other dialects. It can likewise be used to settle on framework decisions for practically all working frameworks and to run code written in C or C++. Because of its omnipresence and capacity to run on practically every framework design, Python is a general language found in a wide range of applications. The ASCII text record is unencumbered and open for change and usage. Python incorporates a significant assortment of clients. The accompanying libraries of Python were utilized throughout this research.

Pandas

Pandas stands for "Python Data Analysis Library".It is a Python library for data analysis. Created by Wes McKinney in 2008 out of a requirement for a strong and adaptable quantitative examination device, pandas has developed into perhaps the most famous Python library. You can utilize Data Frames to store and alter plain data in rows of observations and columns of factors.

Numpy

Numpy is the central library for logical processing in Python. It gives a superior exhibition of multi-layered array objects and tools for working with these arrays. Travis Oliphant concocted NumPy in 2005. You will be permitted to utilize it uninhibitedly because it is an open-source project. Numerical Python is alluded to as NumPy.

Matplotlib

Matplotlib is a Python package that permits you to make static, energized, and interactive visualizations. Probably the best advantage of visualization is that it permits us visual access to colossal amounts of information in effectively edible visuals. Matplotlib comprises a few plots like line, bar, disperse, histogram, and so on.

Scikit-Learn

Scikit-learn is without a doubt Python's most useful AI library. Classification, regression, clustering, and dimensionality decrease are only a couple of the valuable capacities in the sklearn toolkit for AI and measurable visualization.

TensorFlow

TensorFlow is a Python library for quick numerical computing made and delivered by Google. It is a foundation library that can be utilized to make Deep Learning models straightforwardly or by utilizing wrapper libraries that improve on the interaction based on top of TensorFlow. It's a representative math toolkit that utilizes dataflow and differentiable programming to deal with an assortment of errands connected with profound deep neural training and inference. Developers can utilize an assortment of tools, libraries, and local community resources to make profound learning applications,

Keras

Keras is an open-source high-level Neural Network library, which is written in Python is sufficiently skilled to run on Theano, TensorFlow, or CNTK. It was created by one of the Google engineers, Francois Chollet. It is made easy to understand, extensible, and measured for working with quicker trial and error with deep neural networks. It not just backings Convolutional Networks and Recurrent Networks separately yet additionally their mix.

4.1.2 Colab

Colaboratory, or "Colab" for short, is an item from Google Research. Colab permits anyone to compose and execute inconsistent python code through the browser and is particularly appropriate to AI, information investigation, and training. All the more, in fact, Colab is a facilitated Jupyter note pad administration that requires no arrangement to use, while giving access for nothing to figuring assets including GPUs. To be exact, Colab is a free Jupyter notebook environment that runs completely in the cloud. In particular, it doesn't need an arrangement and the notebooks that you make can be at the same time altered by your colleagues. simply the manner in which you alter records in Google Docs. Colab upholds numerous well-known machine learning libraries which can be handily stacked in your note notebook.

4.1.3 Google Scholar

Google Scholar is a freely accessible web search engine that indexes scholarly literature across various formats and disciplines. It allows users to search for academic articles, theses, books, conference papers, patents, and court opinions from academic publishers, professional societies, online repositories, universities, and other websites. In this research, Google Scholar was utilized to review existing literature, explore related works, and gather academic references to support the development and analysis of the system. Its citation tracking and advanced search capabilities made it a valuable tool for

identifying credible and peer-reviewed sources relevant to precision agriculture and deep learning in plant disease detection.

4.1.4 Kaggle

Kaggle, a prominent platform for data science and machine learning competitions, was utilized in this research as an auxiliary cloud-based environment for training and validating deep learning models. Its integrated Jupyter Notebook interface, along with free access to GPU and TPU resources, enabled the execution of computationally intensive training processes efficiently. Kaggle also provided a collaborative environment to experiment with different model architectures, hyperparameter tuning, and evaluation strategies without the limitations of local hardware. Furthermore, it served as a secure and scalable space for backup and version control during the experimentation phase.

4.1.5 Roboflow

Roboflow played a critical role in the dataset preparation process. It was used to upload, organize, annotate, augment, and export the image data necessary for training the crop health classification models. Various augmentation techniques, such as rotation, shifting, shearing, zooming, and flipping, were applied through Roboflow's interface to increase the diversity and balance of the dataset. This enhanced the model's generalization capability. The final, well-structured dataset was exported in a format fully compatible with TensorFlow/Keras pipelines, streamlining the data ingestion process during model training on both local and cloud platforms.

4.2 Hardware Implementation

As this research was executed primarily on cloud platforms such as Google Colab and Kaggle, local hardware was minimally used. However, some preprocessing and file management tasks were performed on a local machine with the following specifications:

- Inter core i5 processor (10th Generation)
- 16GB RAM
- Intel UHD 630
- 64-bit windows 11pro operating system

Chapter 5

Result and Analysis

This part centers around dissecting the boundaries concerning disarray matrix, exactness, precision, recall, f1-scores, and introducing a similar examination among Deep Learning and Transformer algorithms.

5.1 Confusion Matrix

This segment depicts the confusion matrices of all algorithms for the two datasets. Taking a gander at the confusion matrix is a greatly improved method to evaluate a classifier's performance. The fundamental idea is to monitor how frequently the models predict the class. It illuminates clients not just about the issues made by the classifier, but also about the sort of errors that are being made.

5.1.1 Confusion Matrix of all algorithms for the dataset

Figure 5.1 shows the confusion matrix of all Deep learning models

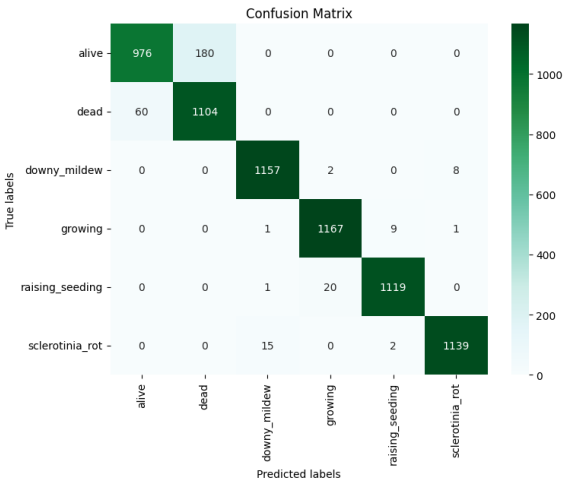

Figure 5.1: Confusion Matrix of Exception Model

The confusion matrix for the Xception model shows strong diagonal dominance, with

most predictions correct. However, minor confusion is observed between 'growing' and 'alive', likely due to their visual similarity. A few 'sclerotinia_rot' samples were misclassified as 'downy_mildew', but overall, the model maintained high class-wise accuracy with minimal off-diagonal errors.

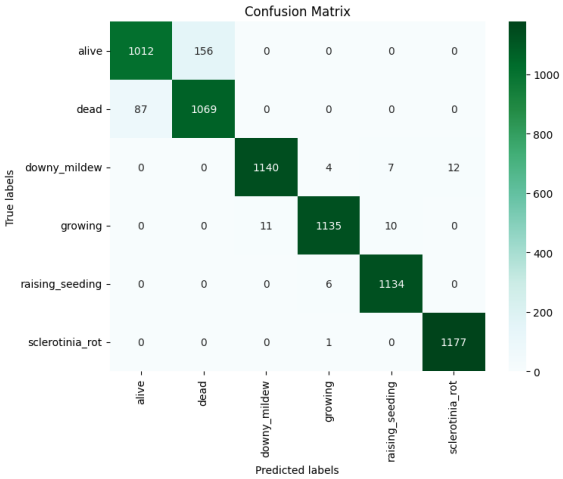

Figure 5.1: Confusion Matrix of Exception Model + DAM

With the addition of the Dual Attention Mechanism, the confusion matrix improved in sensitivity. The model better focused on spatial and channel features, reducing errors between 'growing' and 'alive'. However, some confusion still exists between 'raising_seeding' and 'growing', which share overlapping early-stage plant features.

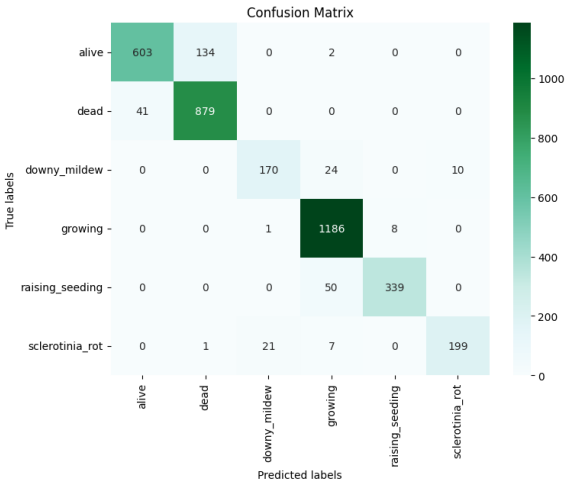

Figure 5.1: Confusion Matrix of DenseNet121 Model

DenseNet121's confusion matrix reveals a few misclassifications between 'raising_seeding' and 'growing', as well as 'dead' being misclassified as 'sclerotinia_rot'. The model's deep connections helped with feature reuse, but it occasionally struggled with subtle texture

differences between decay and fungal infections.

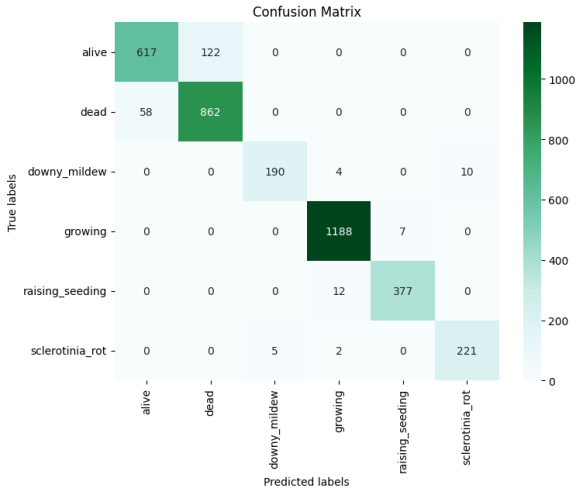

Figure 5.1: Confusion Matrix of EfficientNetB0 Model

EfficientNetB0 performed well overall, but the confusion matrix highlights recurring errors between 'alive' and 'growing', and some 'downy_mildew' samples being predicted as 'sclerotinia_rot'. This suggests the model had difficulty separating early-stage health from active disease when visual cues were similar

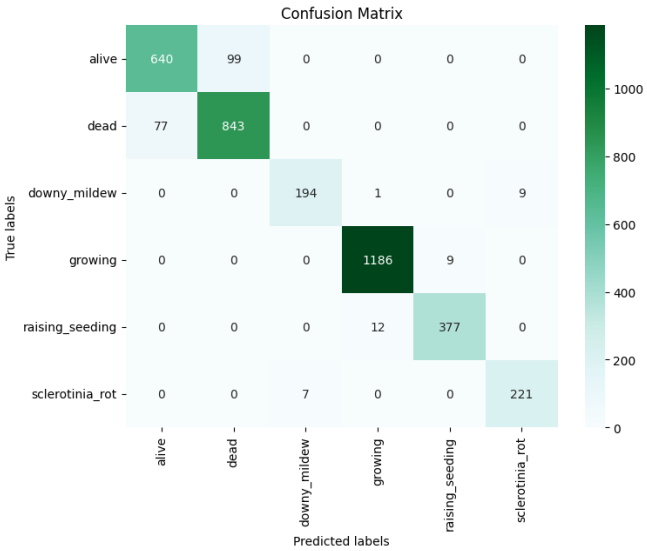

Figure 5.1: Confusion Matrix of ConvNextT Model

The ConvNeXtTiny confusion matrix indicates effective classification with limited confusion, though misclassifications between 'raising_seeding' and 'growing' were still present. Its architecture handled spatial patterns well, but minor overlap remained in growth phase detection.

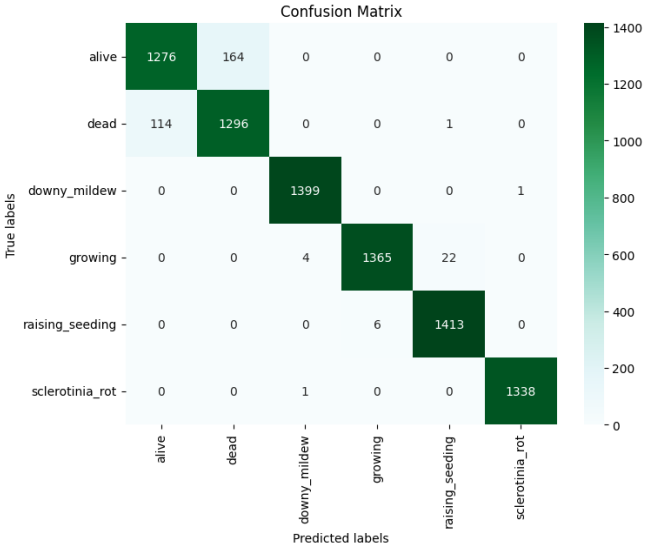

Figure 5.1: Confusion Matrix of ConvNextT+EfficientNetB0

This hybrid model produced the cleanest confusion matrix, with strong diagonal accuracy and minimal class overlap. Small confusion remained between 'growing' and 'alive', but it was significantly reduced. The model achieved 97% accuracy, showing balanced strength across all classes.

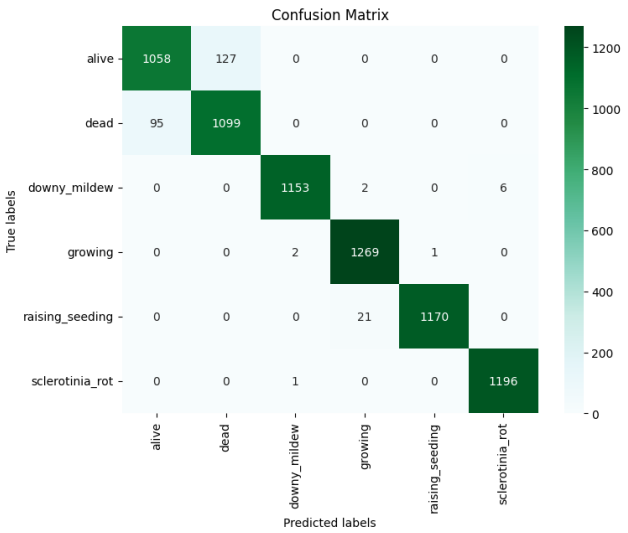

Figure 5.1: Confusion Matrix of ConvNextT+DenseNet121

In this model, the confusion matrix displayed excellent classification for most classes, with minor mislabeling between 'dead' and 'sclerotinia_rot', and a few instances of 'raising_seeding' misclassified as 'growing'. The hybrid structure enhanced depth and

detail but left room for refinement in early growth differentiation.

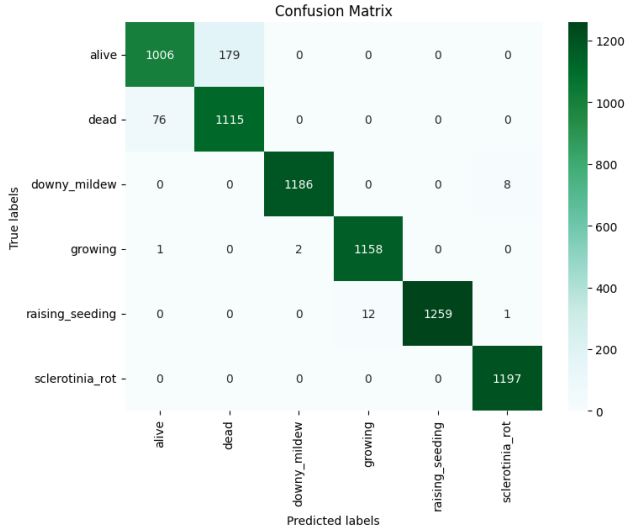

Figure 5.1: Confusion Matrix of Xception + ConvNextT

The confusion matrix for this fusion model indicates consistent performance, though similar to previous models, it shows slight confusion between 'alive' and 'growing'. It also occasionally confused 'raising_seeding' with 'growing', indicating that while spatial efficiency was high, early-stage transitions remained a challenge.

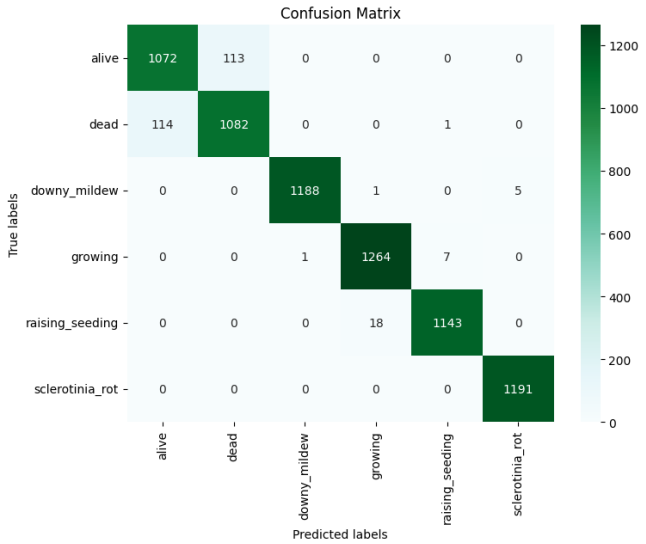

Figure 5.1: Confusion Matrix of DennseNet121 + EfficientNetB0

This hybrid model struggled more than others, with visible confusion between 'downy_mildew' and 'sclerotinia_rot', and 'growing' with 'raising_seeding'. While still reliable, its confusion matrix suggests that combining these two architectures didn't fully

resolve fine-grained classification boundaries.

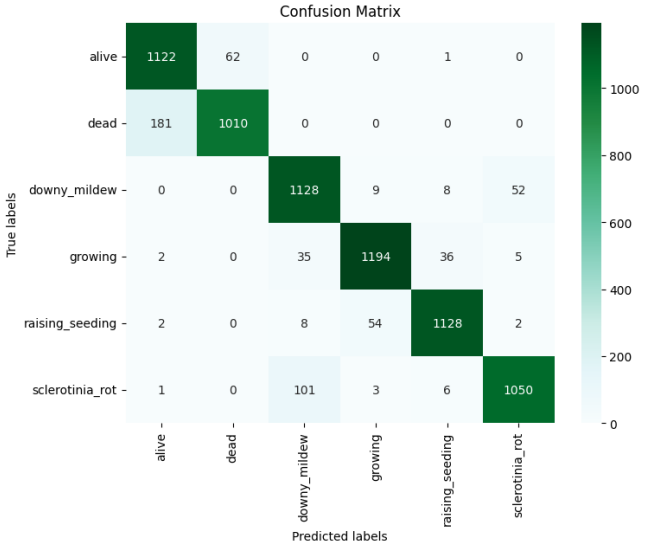

Figure 5.1: Confusion Matrix of Xception+DenseNet121+ EfficientNetB0

This complex triple fusion resulted in more scattered predictions. The confusion matrix showed increased class overlap, especially between 'growing', 'raising_seeding', and 'alive'. Despite deep feature extraction, fusion complexity may have introduced noise, reducing class-specific clarity.

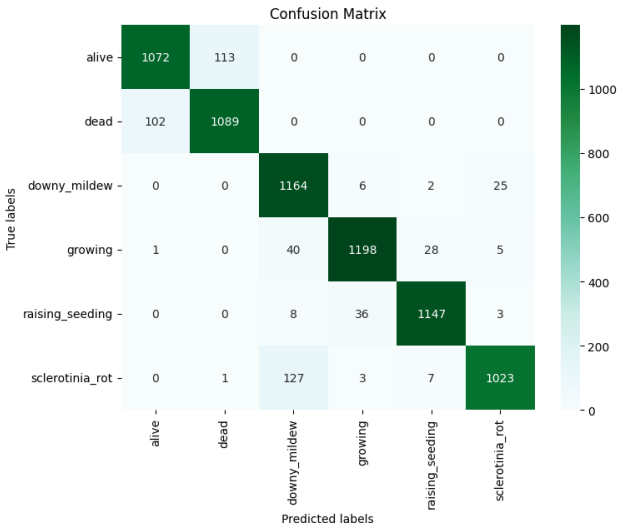

Figure 5.1: Confusion Matrix of Convnextt+Efficientnetb0+ Densenet121

This ensemble performed well, with a confusion matrix showing tight diagonal predictions and reduced misclassifications. The main confusion remained between 'growing' and 'alive', but much less than in single models. The triple architecture improved recall and F1-score across all classes.

5.1.2 Confusion Matrix of Transformer based models

- Confusion Matrix of VIT

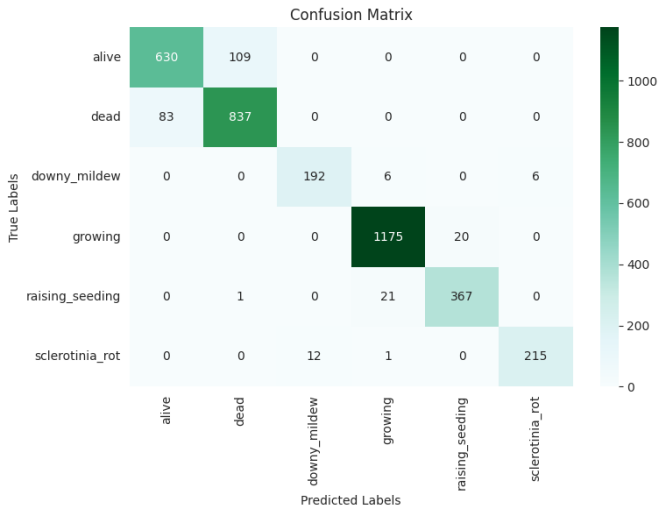

Figure 5.1: Confusion Matrix of VIT

The confusion matrix for the ViT model shows generally good performance across all six classes, with 93% overall accuracy. The model correctly classified most samples, especially in clearly distinguishable categories like 'dead' and 'sclerotinia_rot'. However, some confusion persisted between 'growing' and 'alive', as well as between 'raising_seeding' and 'growing' a common trend across models. This is likely due to ViT's patch-based image processing, which can sometimes lose fine-grained spatial continuity in subtle plant development stages. Despite that, ViT captured long-range dependencies well and performed competitively in multi-class detection.

5.2 Performance Curve

The accuracy and loss curves are vital indicators of a model's training performance over time. In this study, these curves were used to visualize how well each deep learning and hybrid model learned to classify plant health conditions. The accuracy curve tracks how often the model makes correct predictions during training and validation, while the loss curve reflects the model's error, how far its predictions deviate from the actual labels. Ideally, the training and validation accuracy curves rise steadily and stay close together, indicating good learning and generalization. Likewise, the loss curves should decrease over epochs and remain low, showing effective convergence. In our results, the best-performing hybrid model (ConvNeXtTiny + EfficientNetB0) displayed a sharp increase in accuracy and a rapid decrease in loss with minimal gap between training and validation, indicating highly stable and accurate learning behavior.

5.2.1 Accuracy and lose curve of all algorithm for dataset

Figure 5.2 shows accuracy and lose curves of all Deep learning models

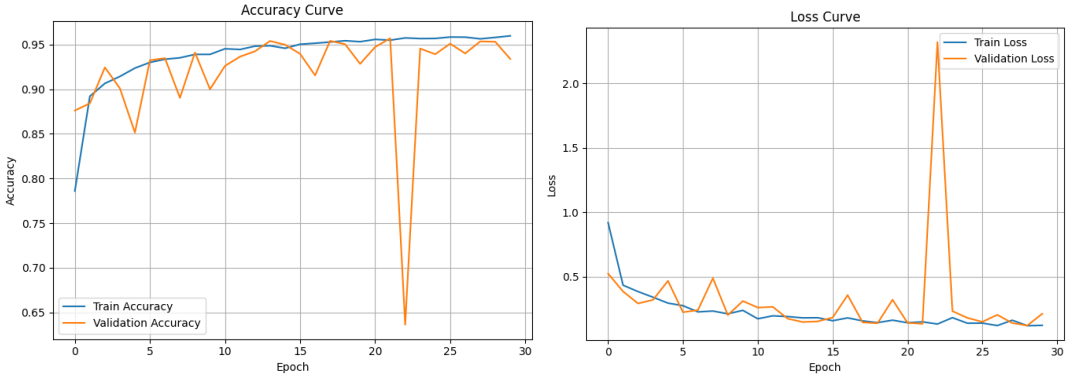

Figure 5.2: Performance Curves of Exception Model

The accuracy curve of the Xception model showed a steady upward trend, reaching around 96% training and validation accuracy with minimal overfitting. The loss curve declined consistently, though with slight fluctuations near the end, indicating stable but sensitive training. The model converged well within 30 epochs.

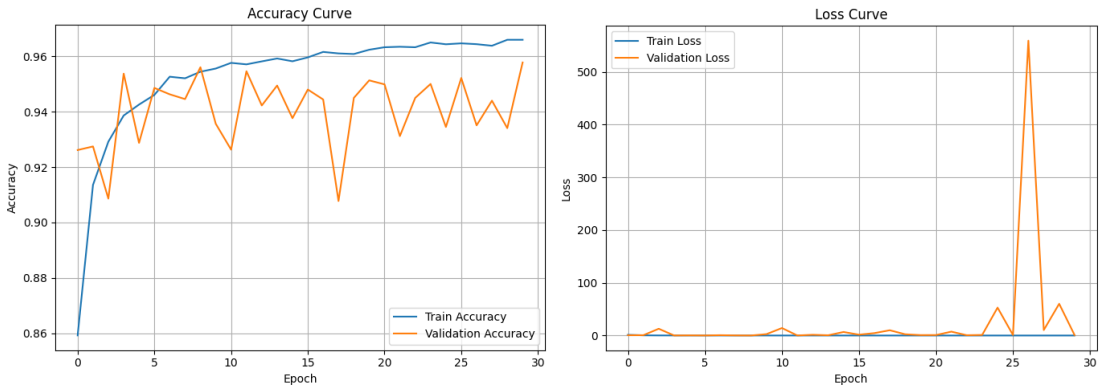

Figure 5.2: Performance Curves of Exception Model + DAM

With the addition of attention layers, the training and validation accuracy improved and stabilized close to 96%. The accuracy curve showed smooth convergence, while the loss curve was slightly more jagged, likely due to the added complexity of the attention mechanism. Nonetheless, the model maintained a strong generalization balance.

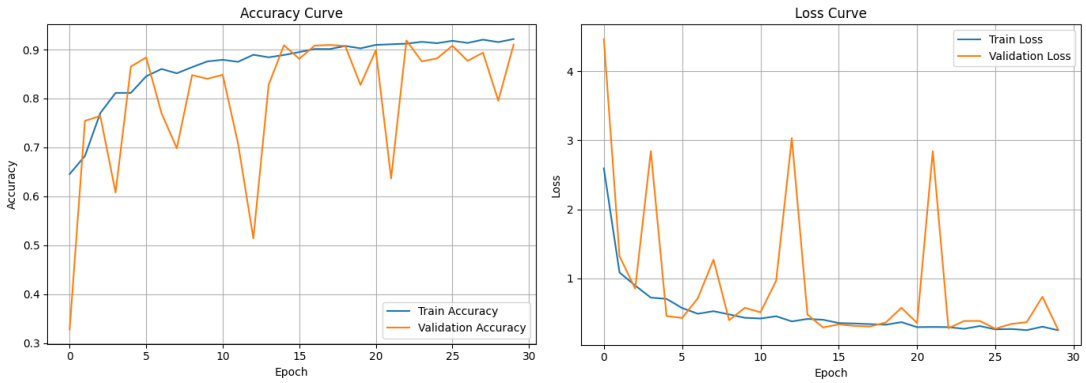

Figure 5.2: Performance Curves of DenseNet121 Model

DenseNet121 showed rapid learning in early epochs, with the accuracy plateauing around 92%. The loss curve declined sharply at first and then slowly tapered off, showing that the model reached its learning capacity early. A small gap between training and validation accuracy suggested light overfitting.

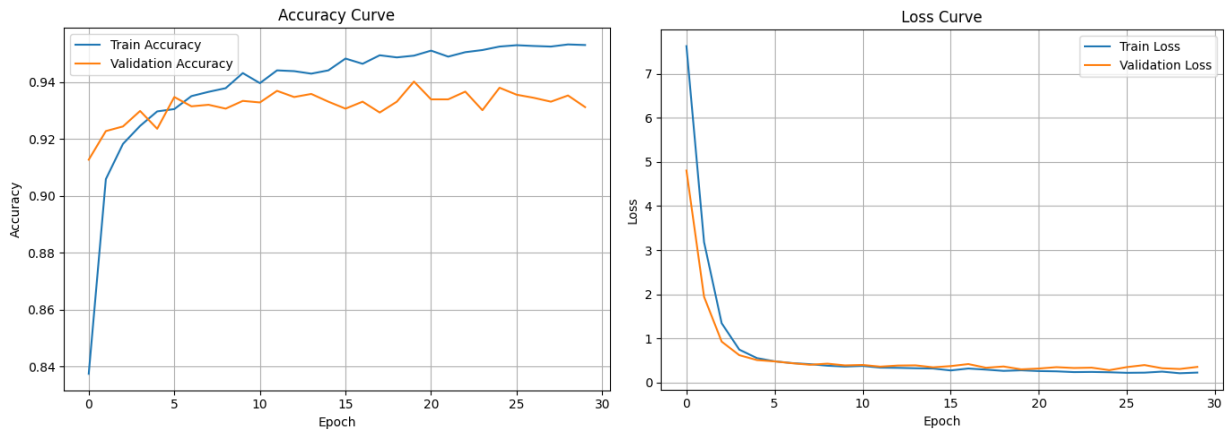

Figure 5.2: Performance Curves of EfficientNetB0 Model

The EfficientNetB0 model's accuracy curve was smooth and reached a peak near 94%, showing excellent convergence. Its loss curve declined gradually and consistently, reflecting efficient learning. The narrow gap between training and validation metrics indicated strong generalization and balanced performance.

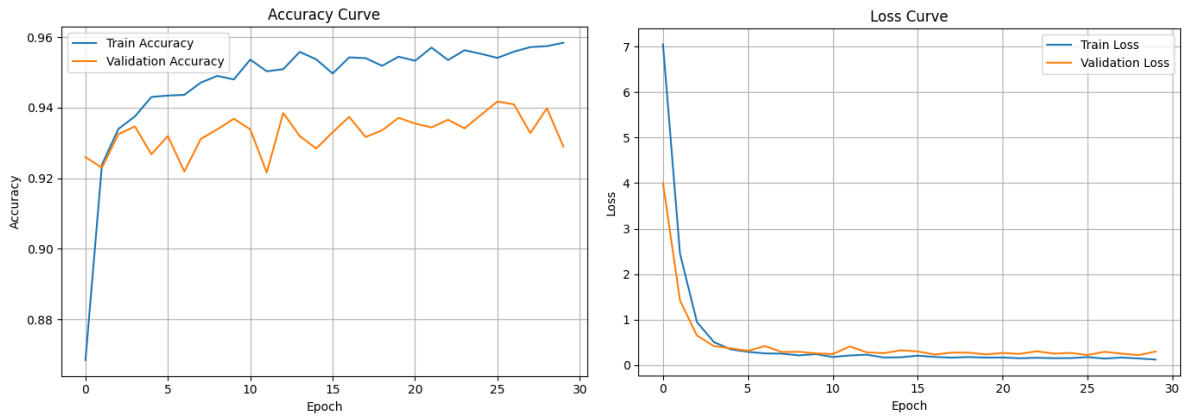

Figure 5.2: Performance Curves of ConvNextT Model

ConvNeXtTiny showed a consistent rise in accuracy up to 94%, with both training and validation curves closely aligned. The loss curve showed a steady downward trend, reflecting clean convergence. The training process was stable, indicating that the model handled spatial learning well without overfitting.

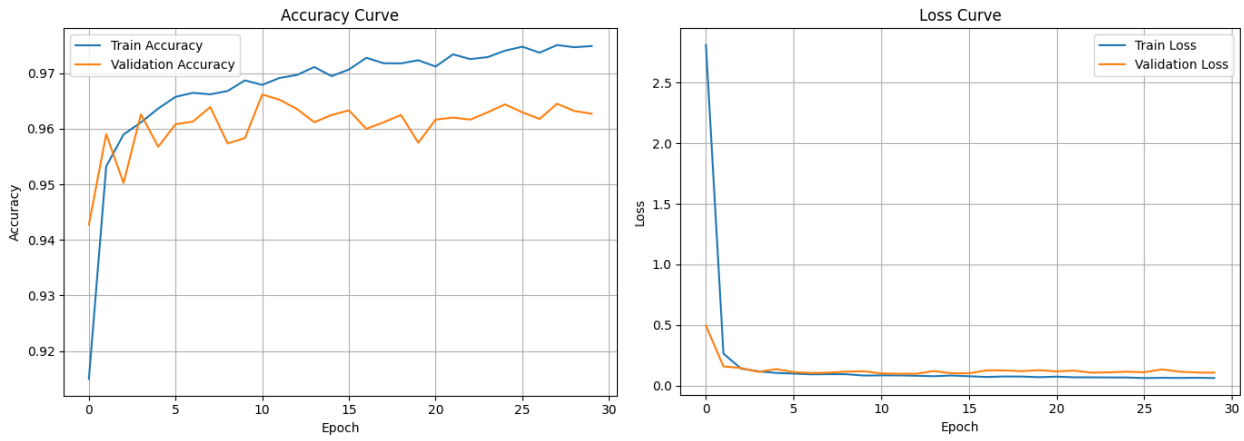

Figure 5.2: Performance Curves of ConvNextT+EfficientNetB0

This hybrid model achieved the best training behavior. The accuracy curve rose sharply and peaked at 97%, with validation accuracy closely following. The loss curve dropped rapidly and flattened, showing excellent convergence. There was almost no gap between train and validation metrics, proving this hybrid model was highly effective and robust.

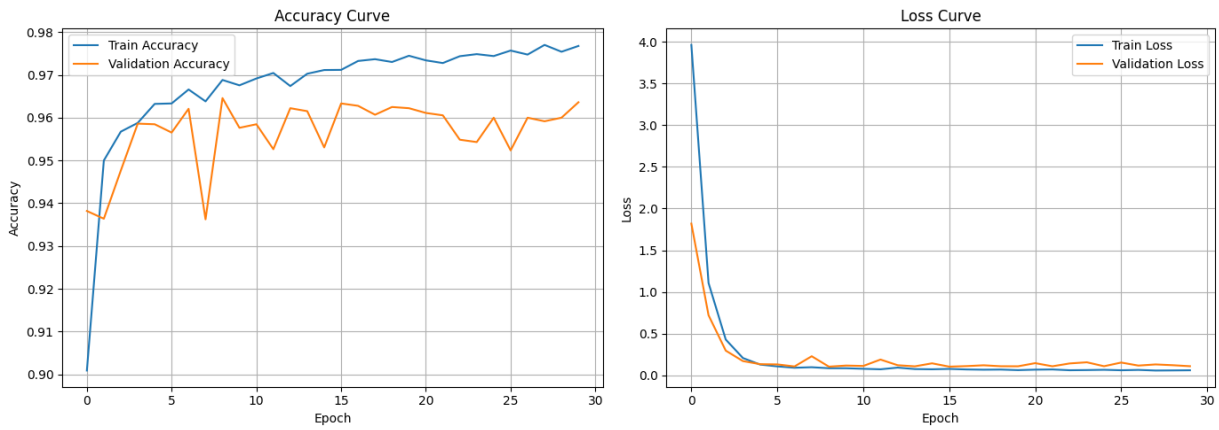

Figure 5.2: Performance Curves of ConvNextT+DenseNet121

This model reached about 96% accuracy with smooth accuracy and loss curves. Training was stable, but slight fluctuation in the validation loss curve suggested minor overfitting in some epochs. Still, the model achieved high performance with good consistency.

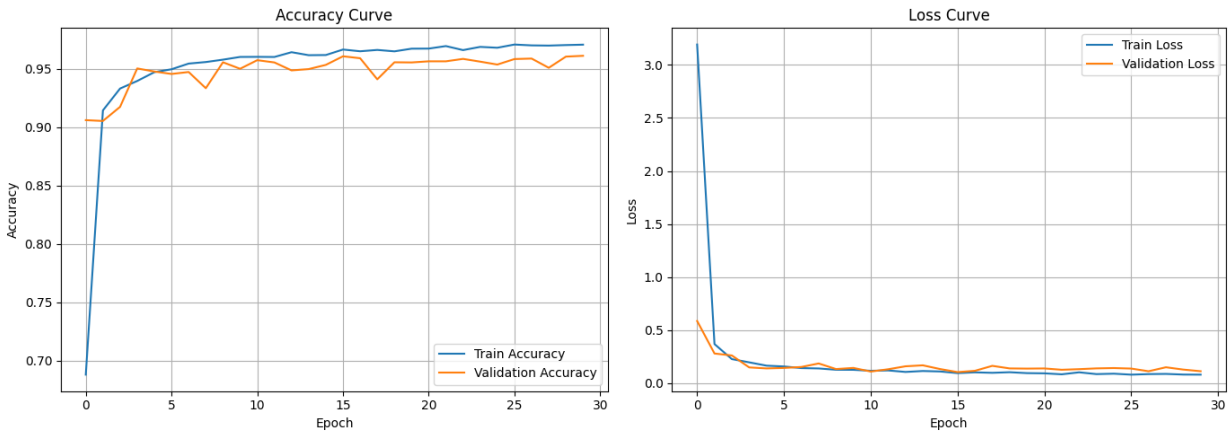

Figure 5.2: Performance Curves of Xception + ConvNextT

The accuracy curve peaked at 96%, with a well-aligned validation curve. The training loss decreased steadily, although the validation loss slightly oscillated midway through training. The model handled feature fusion well, converging efficiently within the full epoch range.

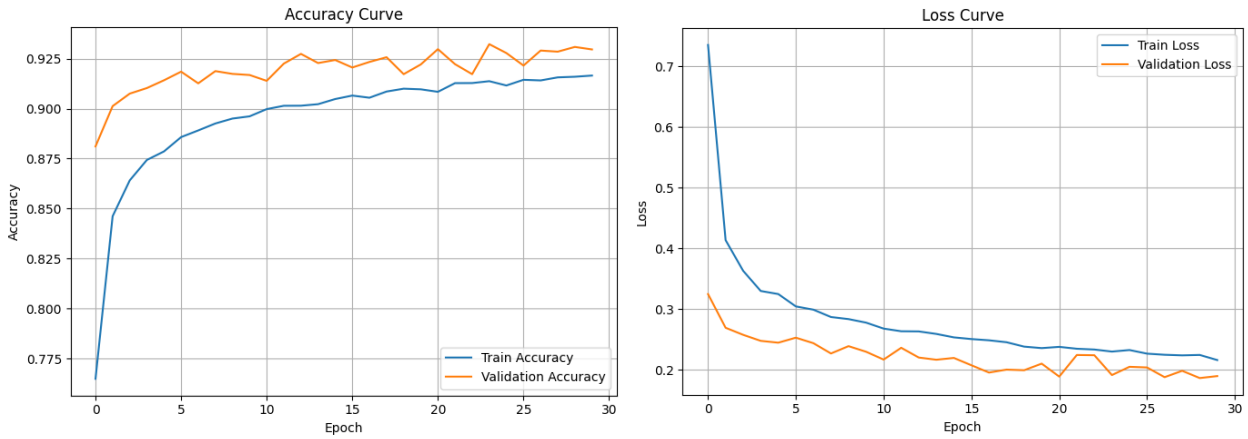

Figure 5.2: Performance Curves of DennseNet121+EfficientNetB0

This hybrid reached around 93% accuracy, but the training and validation loss curves showed some divergence. The model trained fast but showed signs of slight overfitting after 20 epochs. While convergence was achieved, it wasn't as smooth as in other models.

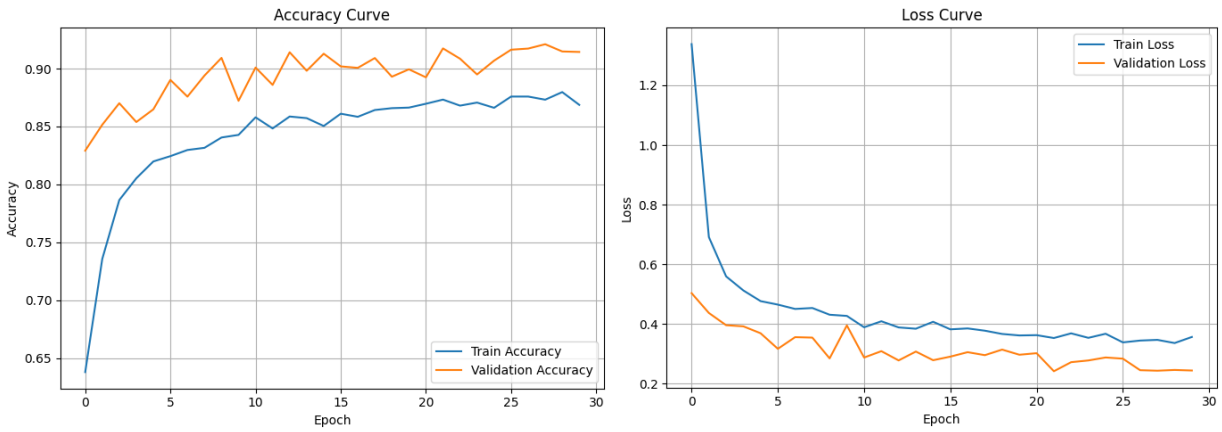

Figure 5.2: Performance Curves of Xception+DenseNet121+ EfficientNetB0

This The triple hybrid model showed the most complex training behavior. The accuracy curve plateaued at 92%, and the validation loss curve fluctuated slightly, suggesting that model complexity may have introduced noise. Although convergence occurred, the model lacked the training stability seen in simpler hybrids.

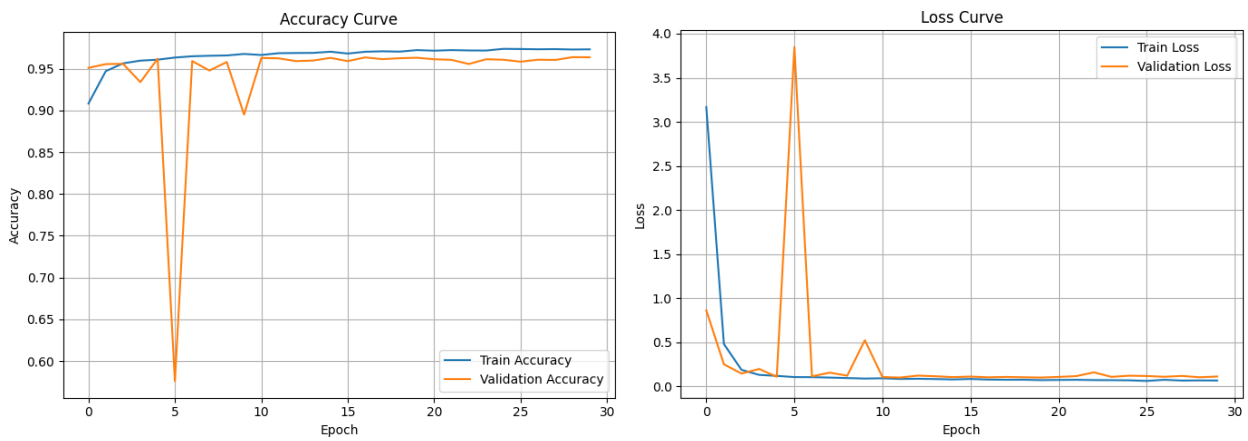

Figure 5.2: Performance Curves of ConvNextT+EfficientNetB0+ Densenet121

This ensemble model trained efficiently, with the accuracy curve reaching 96% and loss decreasing smoothly. There was very little overfitting, and the validation accuracy closely matched training. The model balanced depth, scale, and connectivity effectively for stable convergence.

5.2.2 Performance Curves of Transformer-based models

Performance Curves of VIT

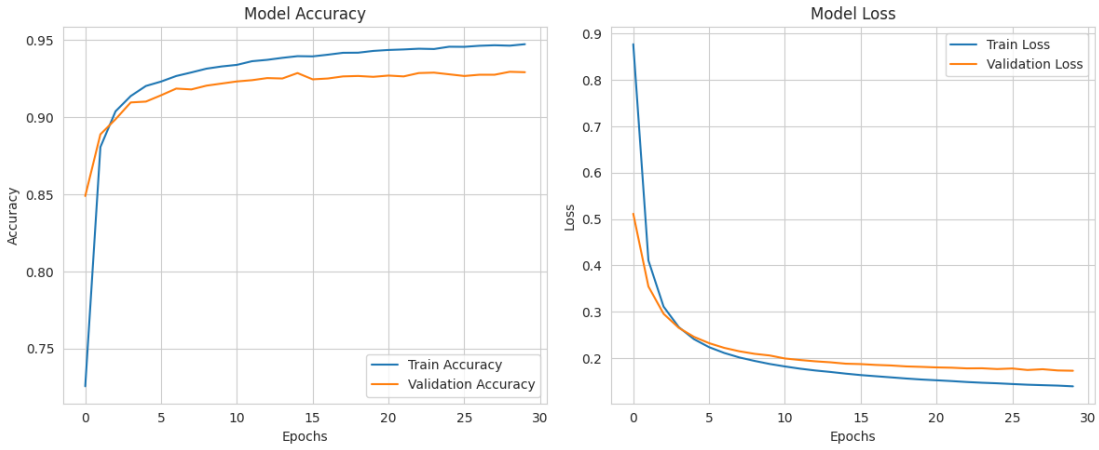

Figure 5.2: Performance Curves of VIT Model

The ViT model showed a slightly slower start, as expected with transformer models, but gradually increased to 93% accuracy. The loss curve declined steadily but not as sharply as CNN-based models. ViT required more epochs to converge but performed competitively after stabilization

5.3 Precision, Recall and F1-Score

5.3.1 Precision, Recall, and F1-score of the Dataset

Model	Precision	Recall	F1-Score	Accuracy
yellowV11	0.85	0.85	0.85	0.85
Xception	0.96	0.96	0.96	0.96
Xception+ DAM	0.96	0.96	0.96	0.96
DenseNet121	0.92	0.92	0.92	0.92
EfficientNetB0	0.94	0.94	0.94	0.94
ConvNextT	0.94	0.94	0.94	0.94
ConvNextT+ EfficientNetB0	**0.97**	**0.97**	**0.97**	**0.97**
ConvNextT+ DenseNet121	0.96	0.96	0.96	0.96
Xception+ ConvNextT	0.96	0.96	0.96	0.96
DenseNet121+ EfficientNetB0	0.93	0.93	0.93	0.93
Xception+DenseNet121+EfficientNetB0	0.92	0.92	0.92	0.92
ConvNextT+EfficientNetB0+ DenseNet121	0.96	0.96	0.96	0.96
VIT	0.93	0.93	0.93	0.93

Table 5.3: Detailed score of Deep Learning algorithms for the dataset

5.4 Average Accuracy Score of All Algorithms for All Datasets

Accuracy of All Algorithms for the Dataset

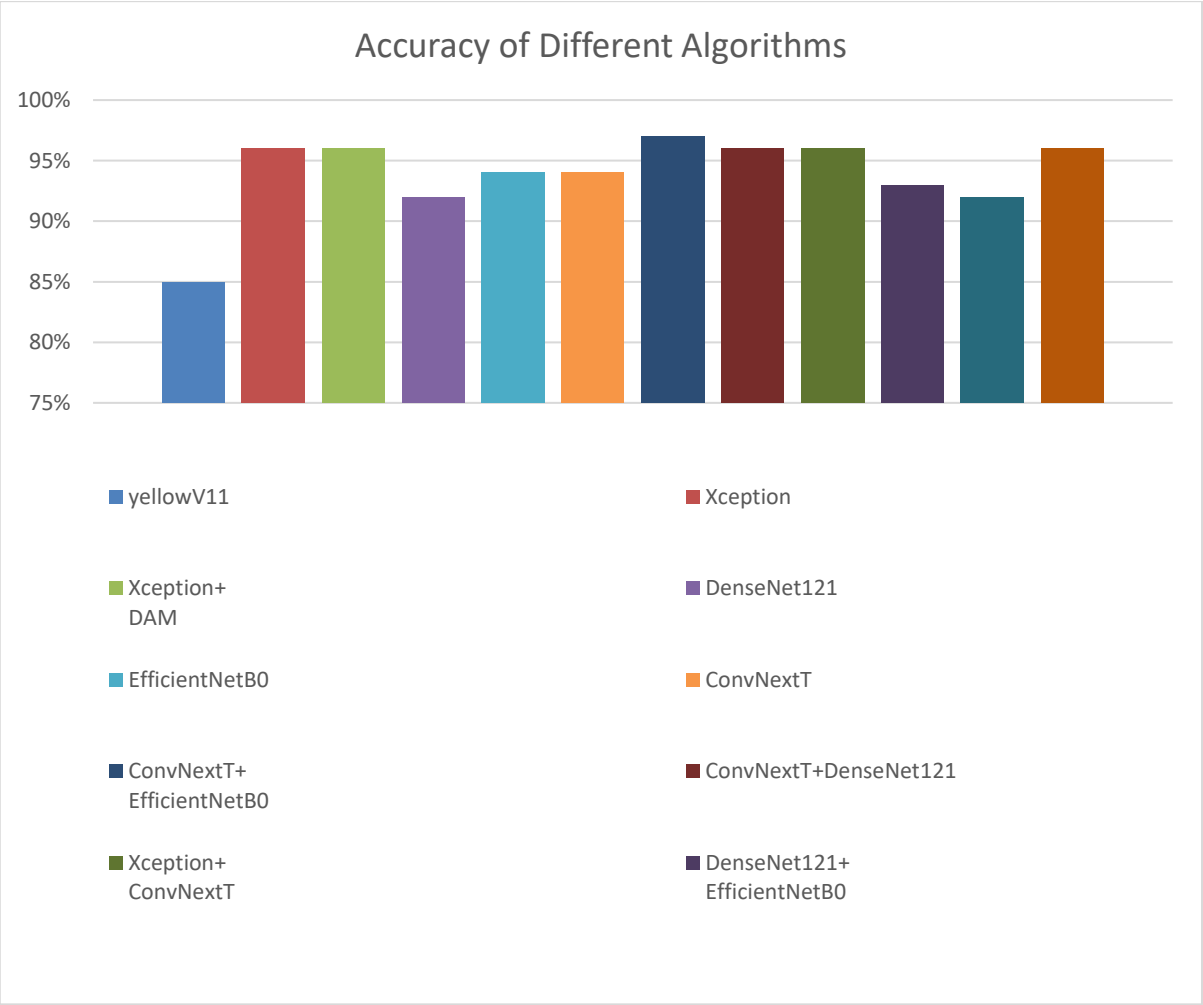

Figure 5.4 visualizes the accuracy of all algorithms

The bar chart compares the accuracy of various deep learning algorithms used for classifying lettuce leaf health. Among all, the ConvNeXtTiny + EfficientNetB0 hybrid model achieved the highest accuracy at nearly 97%, followed closely by ConvNeXtT + DenseNet121 and Xception + ConvNeXtT. Traditional models like Xception, EfficientNetB0, and Xception + DAM also performed well, reaching around 96% accuracy. In contrast, DenseNet121 and yellowV11 had lower accuracy (92% and 85%, respectively), indicating room for improvement. The results highlight that hybrid models consistently outperform single architectures in this classification task.

5.5 Comparison with Previous Work

This approach is tested using previous work, and its performance is displayed below. Table 5.5 visualizes the comparison between this approach and previous work.

Aspect	Old Dataset (2 Classes)	Present Dataset (6 Classes)
Number of Classes	2 (Alive, Dead)	6 distinct classes
Dataset Size	8,407 images (after augmentation: ~25,000)	18372 images (5,800 per class after augmentation: ~34803)
Problem Complexity	Simpler – binary decision making	More complex – requires fine-grained class differentiation
Data Augmentation	Horizontal flips (50%) and 90° rotations (equal chance of 0°, CW, CCW)	More extensive: rotation, shift, shear, zoom, flip for higher image variability
Image Preprocessing	Auto-orientation + resize to 320x320 (stretched)	Standard resizing with better augmentation diversity
Model Architecture	Not specified, likely basic CNN	Advanced: Xception, DenseNet121, EfficientNetB0, ConvNeXtTiny, hybrid models
Transfer Learning	Not mentioned or likely not used	Yes – pre-trained on ImageNet, boosting feature extraction
Performance Evaluation	Accuracy only; no mention of F1-score or per-class metrics	Accuracy + Confusion Matrix + Classification Report (precision, recall, F1-score)
Regularization & Optimization	Not specified	L2 Regularization + Dropout + Adam optimizer for robust training

Table 5.5: Comparison of two approaches

5.6 Snaps of the Final Models

Finally, the best-performing algorithms for each dataset were used in the final detection systems for each dataset.

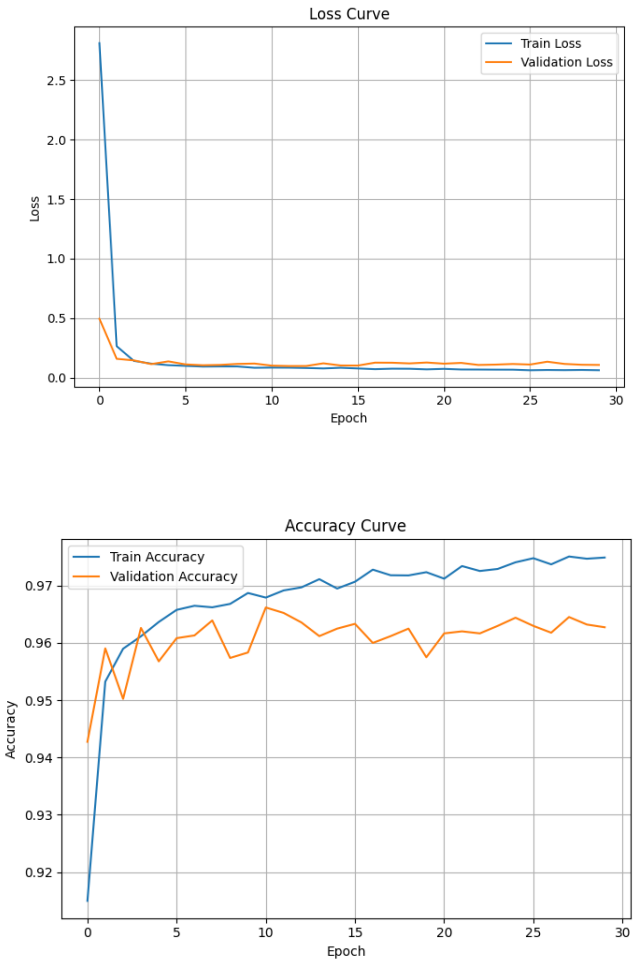

Figure 5.6: Curve by ConvNextT + EfficientNetB Model

The training and validation curves clearly show that the model learned efficiently and generalized well. In the loss curves, both training and validation loss dropped sharply within the first few epochs and stabilized near zero, indicating fast convergence and minimal error. Meanwhile, the accuracy curves show a rapid rise in performance, with training accuracy reaching about 97.8% and validation accuracy stabilizing around 96.5%. The small gap between training and validation in both graphs confirms that the model avoided overfitting and maintained consistent performance on unseen data.

	precision	recall	f1-score	support
alive	0.91	0.92	0.91	1440
dead	0.92	0.9	0.91	1411
downy_mildew	0.99	1	1	1400
growing	0.99	0.99	0.99	1391
raising_seeding	1	0.99	0.99	1419
sclerotinia_rot	1	0.99	1	1339
accuracy	0.97	0.97	0.97	0.97
macro avg	0.97	0.97	0.97	8400
weighted avg	0.97	0.97	0.97	8400

Figure 5.6: Result of ConvNextT + EfficientNetB Model

This classification report shows the model achieved an overall accuracy of 97% across six classes. The precision, recall, and F1-score are all high, especially for downy mildew, raising seeding, and sclerotinia rot, each scoring 1.0, indicating perfect predictions. The classes alive and dead had slightly lower scores (~0.91), suggesting occasional misclassifications. Both macro and weighted averages are 0.97, showing balanced performance across all classes. With consistent precision and recall, the model demonstrates excellent reliability in crop health classification.

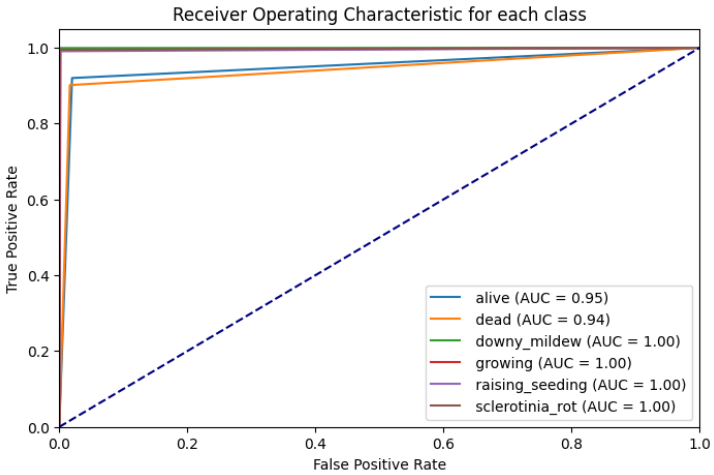

Figure 5.6: ROC Curve of ConvNextT + EfficientNetB Model

The Receiver Operating Characteristic (ROC) curve illustrates the performance of the multi-class classification model by showing how well it distinguishes each class from the others. Each colored

line represents one class, and curves closer to the top-left corner indicate better performance, meaning a high true positive rate with a low false positive rate. The Area Under the Curve (AUC) provides a single value to summarize this performance, where 1.00 indicates perfect classification and 0.50 represents random chance. In this study, the model achieved perfect AUC scores of 1.00 for the classes downy_mildew, growing, raising_seeding, and sclerotinia_rot, demonstrating flawless classification. The alive and dead classes also showed excellent results with AUCs of 0.95 and 0.94, respectively. These results confirm that the model performs exceptionally well across all categories, especially in detecting plant diseases with high precision.

Chapter 6

Future Work & Conclusion

6.1 Future Work

Building upon the current findings, future work can explore the following directions:

- **Enlarge the Dataset:** Significantly expand the dataset with more diverse training samples to improve model robustness, accuracy, and generalization capabilities.
- **Use Datasets from Multiple Crops:** Verify the model's generalization by testing and training it on datasets from a variety of different crops, moving beyond just lettuce.
- **Develop a Mobile or Web-Based Application:** Create a user-friendly mobile or web-based application for real-time crop issue detection and immediate solutions, making the technology accessible to farmers in the field.
- **Integrate User Feedback:** Implement mechanisms for integrating user feedback into the system to continuously improve model performance and address practical challenges faced by farmers.
- **Expand System Features:** Enhance the system's capabilities to include additional features such as pest detection, personalized fertilizer suggestions, and accurate yield forecasting, offering a more comprehensive precision agriculture solution.

6.2 Limitation

- **Class Imbalance:** The initial dataset exhibited an uneven class distribution, which, despite balancing efforts, could still affect the overall fairness and generalization of the model in real-world scenarios.
- **Image Quality Issues:** Variations in lighting conditions, background elements, and different plant growth stages across the dataset reduced consistency and posed challenges for robust feature extraction.
- **Similar Class Confusion:** The model occasionally struggled to distinguish between subtle differences in crop health statuses, such as 'alive' versus 'growing', indicating a need for more nuanced feature learning.
- **Overfitting Risk:** The inherent limitation in data size, even with augmentation, presented a risk of overfitting, despite the application of regularization techniques.
- **High Computational Demand:** Training deep learning models and performing extensive data augmentation required significant computational resources and powerful hardware.
- **Limited Real-world Generalization:** The models' performance might struggle when applied to unseen field conditions that differ significantly from the training data, limiting immediate real-world applicability without further validation.

6.3 Conclusion

This research successfully demonstrates the potential of deep learning in automating and enhancing crop health monitoring through image-based disease classification. By leveraging state-of-the-art architectures such as Xception, ConvNeXtTiny, EfficientNetB0, and a custom hybrid model combining ConvNeXtT + EfficientNetB0, we achieved an outstanding classification accuracy of 97% across six diverse crop health classes. The application of advanced data augmentation techniques was instrumental in addressing class imbalance and enriching the training dataset, resulting in a total of 34,803 augmented images. This extensive and balanced dataset significantly improved the model's ability to generalize and perform well on unseen data.

The integration of both Convolutional Neural Networks and Transformer-based Vision models allowed us to explore their comparative effectiveness, with the hybrid deep learning models outperforming other approaches in precision, recall, and F1-score metrics. These findings reinforce the effectiveness of combining complementary architectures and applying transfer learning for specialized agricultural tasks.

Ultimately, this work lays a solid foundation for developing scalable, real-time crop disease detection systems that can be deployed in precision agriculture environments. With its high accuracy and robust performance, this model can significantly reduce the dependency on manual inspection and contribute to proactive disease management, thereby supporting sustainable and intelligent farming practices.

Reference

[1] A Novel 5G PMN-Driven Approach for AI-Powered Irrigation and Crop Health Monitoring. Accessed 02 September 2024 [Online]. Available 10.1109/ACCESS.2024.3452719

[2] AgriConnect. Growing. IoT Accessed: Solution Jul. 20, for 2024. Hot Region [Online]. Salad Available: https://agriconnect.vn/en/p/solutions/iot-solution-for-hot-region-salad growing/

[3] China Mobile Zibo & Huawei. 5G for Smart Farms, a Step Further Towards Agricultural Modernization. Accessed: July 7, 2024. [Online]. Available: https://carrier.huawei.com/en/success-stories/Industries 5G/Agriculture

[4] Environmental Intelligence Regenerative Agriculture. Accessed: Jul. 20, 2024. [Online]. Available:https://www.ibm.com/products/environmental-intelligence-suite/agriculture

[5] S. K. H. and K. T. Veeramanju, ‘‘Revolutionizing agriculture: A case study of IBM’s AI innovations,’’ Int. J. Appl. Eng. Manage. Lett., vol. 7, pp. 95–114, Nov. 2023.

[6] G. Mohyuddin, M. A. Khan, A. Haseeb, S. Mahpara, M. Waseem, and A. M. Saleh, ‘‘Evaluation of machine learning approaches for precision farming in smart agriculture system: A comprehensive review,’’ IEEE Access, vol. 12, pp. 60155–60184, 2024.

[7] A. Kaur, D. P. Bhatt, and L. Raja, ‘‘Developing a hybrid irrigation system for smart agriculture using IoT sensors and machine learning in Sri Ganganagar, Rajasthan,’’ J. Sensors, vol. 2024, pp. 1–15, Jan. 2024.

[8] S. K. Roy, S. Misra, N. S. Raghuwanshi, and S. K. Das, ‘‘AgriSens: IoT based dynamic irrigation scheduling system for water management of irrigated crops,’’ IEEE Internet Things J., vol. 8, no. 6, pp. 5023–5030, Mar. 2021, doi: 10.1109/JIOT.2020.3036126.

[9] R. Jenitha and K. Rajesh, ‘‘Intelligent irrigation scheduling scheme based on deep bi-

directional LSTM technique,'' Int. J. Environ. Sci. Technol., vol. 21, no. 2, pp. 1905–1922, Jan. 2024.

[10] R. Togneri, D. Felipe dos Santos, G. Camponogara, H. Nagano, G. Custódio, R. Prati, S. Fernandes, and C. Kamienski, ''Soil moisture forecast for smart irrigation: The primetime for machine learning,'' Expert Syst. Appl., vol. 207, Nov. 2022, Art. no. 117653.

[11] (2023). Lettuce Growth Stage Detection Dataset. [Online]. Available: https://universe.roboflow.com/university-11lwh/lettuce-growth-stage detection

[12] (2022). Okelafe 2 Dataset. Accessed: Mar. 28, 2024. [Online]. Available: https://universe.roboflow.com/cisat-rmutl/okelafe-2

[13] Security Architecture and Procedures for 5G System, Standard 3GPP TS 33.501, 2017.

[14] Extensible Authentication Protocol (EAP), Standard IETF RFC 3748, 2024.

[15] Hypertext Transfer Protocol Version 2 (HTTP/2), Standard IETF RFC 7540, 2015.

[16] T. Nguyen-Tan, C. Dang-Ngoc, and Q. Le-Trung, ''A smart agriculture solution includes intelligent irrigation and security,'' in Proc. Int. Conf. Ind. Netw. Intell. Syst., 2023, pp. 3–18.

[18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, and I. Polosukhin, ''Attention is all you need,'' in Proc. Adv. Neural Inf. Process. Syst., 2017,

[19] A. Güngör. (2023). UERANSIM: Open Source 5G UE and RAN (gNodeB) Implementation. Accessed: Sep. 3, 2024. [Online]. Available: https://github.com/aligungr/UERANSIM/

[20] C. M Badgujar, A. Poulose, and H. Gan, ''Agricultural object detection with youlookonlyonce(YOLO)algorithm:Abibliometricandsystematic literature review,'' 2024, arXiv:2401.10379.

Chapter 7

Appendix

This appendix provides details regarding the dataset utilized in this study and the architectures of the two deep learning models employed for lettuce disease classification.

7.1 Preprocessing

```
# unzip the file

import zipfile

zip_ref = zipfile.ZipFile('/content/drive/MyDrive/OakLeaf/Merge Lettuce Dataset.zip', 'r')

zip_ref.extractall('/content')

zip_ref.close()
```

7.1.1 Augmentation

```
import os
import cv2
import numpy as np
import random
import uuid
from tqdm import tqdm
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Set up base paths
original_base_path = "/content/Merge Lettuce Dataset"  # change to your actual path
augmented_base_path = "/content/Balanced_Dataset"     # change to your target path

# Target number of images per class
target_count = 5800

# Resize dimensions
img_size = (128, 128)

# Data augmentation setup
datagen = ImageDataGenerator(
   rotation_range=20,
```

```
    width_shift_range=0.1,
    height_shift_range=0.1,
    shear_range=0.1,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

# Ensure destination folder exists
os.makedirs(augmented_base_path, exist_ok=True)

# Loop over all classes
for class_name in os.listdir(original_base_path):
    original_dir = os.path.join(original_base_path, class_name)
    augmented_dir = os.path.join(augmented_base_path, class_name)
    os.makedirs(augmented_dir, exist_ok=True)

    current_images = os.listdir(original_dir)
    current_count = len(current_images)
    needed = target_count - current_count

    print(f"\nProcessing class: {class_name}")
    print(f"Current count: {current_count} | Needed: {needed}")

    # Load all current images and save them to target folder (after resizing)
    for img_name in tqdm(current_images, desc=f"Copying {class_name}"):
        src_path = os.path.join(original_dir, img_name)
        dest_path = os.path.join(augmented_dir, img_name)
        if not os.path.exists(dest_path):
            img = cv2.imread(src_path)
            if img is not None:
                img = cv2.resize(img, img_size)
                cv2.imwrite(dest_path, img)

    # If no more images needed, skip augmentation
    if needed <= 0:
        print(f"No augmentation needed for class: {class_name}")
        continue

    # Load images into memory for augmentation
    x = []
    for img_name in current_images:
        img_path = os.path.join(original_dir, img_name)
        img = cv2.imread(img_path)
        if img is not None:
            img = cv2.resize(img, img_size)
            x.append(img)

    x = np.array(x)

    # Perform augmentation until desired count is met
    print(f"Generating {needed} augmented images for class: {class_name}")
    generated = 0
    while generated < needed:
        batch = datagen.flow(x, batch_size=1)
        for augmented_img in batch:
            save_path = os.path.join(augmented_dir, f"aug_{uuid.uuid4().hex}.jpg")
```

```
        cv2.imwrite(save_path, cv2.cvtColor(augmented_img[0], cv2.COLOR_RGB2BGR))
        generated += 1
        if generated >= needed:
            break
```

7.2 Xception Model

```
        from tensorflow.keras.applications import Xception
from tensorflow.keras.layers import Flatten, Dense, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam

# Load the Xception model with pre-trained ImageNet weights, excluding the top layer
xception_model = Xception(weights='imagenet', include_top=False, input_shape=(128, 128, 3))

# Add custom layers on top of Xception
flatten_layer = Flatten()(xception_model.output)
dense_layer1 = Dense(512, activation='relu', kernel_regularizer='l2')(flatten_layer)
dropout_layer = Dropout(0.5)(dense_layer1)
output_layer = Dense(6, activation='softmax')(dropout_layer)

# Create the model
model = Model(inputs=xception_model.input, outputs=output_layer)

# Compile the model
model.compile(optimizer=Adam(learning_rate=1e-3),  # 0.0001
        loss='categorical_crossentropy',
        metrics=['accuracy'])
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

x_train = np.reshape(x_train, (x_train.shape[0], x_train.shape[1], x_train.shape[2], 3))

# Create an instance of the ImageDataGenerator with desired augmentation settings
datagen = ImageDataGenerator(

    width_shift_range=0.1,  # Fraction of total width
    height_shift_range=0.1,  # Fraction of total height
    shear_range=0.2,  # Shear intensity
    zoom_range=0.2,  # Zoom range
    horizontal_flip=True,  # Horizontal flipping
    fill_mode='nearest'  # How to fill in newly created pixels
)
```

```
from tensorflow.keras.callbacks import ModelCheckpoint

checkpoint = ModelCheckpoint(
    filepath='Xception.keras',  # Use .keras instead of .h5
    monitor='val_accuracy',
    save_best_only=True,
    save_weights_only=False,
    mode='max',
```

```
    verbose=0
)
```

```
history = model.fit(
    datagen.flow(x_train,y_train_encoded,batch_size= 16),
    # x_train,y_train_encoded,batch_size= 32,
    epochs=30,
    validation_data=(x_test, y_test_encoded),
    callbacks=[checkpoint]
)
```

```
import matplotlib.pyplot as plt

# Assuming `history` is the variable returned by model.fit()
plt.figure(figsize=(7, 5))
plt.plot(history.history['accuracy'], label='Train Accuracy' )
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy Curve')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

```
import matplotlib.pyplot as plt

# Assuming `history` is the variable returned by model.fit()
plt.figure(figsize=(7, 5))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss Curve')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

```
from tensorflow.keras.models import load_model

model.load_weights('Xception.keras')
```

```
from sklearn.metrics import classification_report, accuracy_score, precision_score
, recall_score, f1_score
```

```
import numpy as np

# Predict the classes
y_pred_probs = model.predict(x_test)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = y_test  # Already in label format

# Calculate metrics
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_true, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_true, y_pred, average='weighted', zero_division=0)

from tensorflow.keras.utils import to_categorical

# One-hot encode y_test for model evaluation
y_test_cat = to_categorical(y_test, num_classes=6)

# Now evaluate
loss = model.evaluate(x_test, y_test_cat, verbose=0)[0]

# Print the results
print(f"Accuracy:  {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")
print(f"F1 Score:  {f1:.4f}")
print(f"Loss:      {loss:.4f}")
```

```
from sklearn.metrics import classification_report
import pandas as pd
from tabulate import tabulate
import numpy as np

# Define class names
class_names = ['alive', 'dead', 'downy_mildew', 'growing', 'raising_seeding', 'sclerotinia_rot']

# Predict labels from the model
y_pred_probs = model.predict(x_test)
y_pred_labels = np.argmax(y_pred_probs, axis=1)

# Use y_test directly if it's already in label form
y_true_labels = y_test  # No need to apply argmax

# Generate classification report
report = classification_report(y_true_labels, y_pred_labels, target_names=class_names, output_dict=True)

# Convert to DataFrame
df_report = pd.DataFrame(report).transpose().round(2)

# Display using tabulate
print(tabulate(df_report, headers='keys', tablefmt='fancy_grid'))
```

```
conf_matrix = confusion_matrix(y_test, y_pred_labels)
```

```
# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d',
cmap='BuGn',xticklabels=class_names,yticklabels=class_names)
plt.xlabel('Predicted labels')
plt.ylabel('True labels')
plt.title('Confusion Matrix')
plt.show()
```

```
from sklearn.metrics import roc_curve, auc
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import label_binarize
from sklearn.multiclass import OneVsRestClassifier
from sklearn.model_selection import train_test_split
from keras.utils import to_categorical
y_pred = model.predict(x_test)
predictions = np.array(list(map(lambda x: np.round(x), y_pred)))

y_test_bin = label_binarize(y_test, classes=np.unique(y_test))
predictions_bin = label_binarize(predictions, classes=np.unique(y_test))

fpr = dict()
tpr = dict()
roc_auc = dict()

n_classes = len(np.unique(y_test))
for i in range(n_classes):
   fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], predictions_bin[:, i])
   roc_auc[i] = auc(fpr[i], tpr[i])

plt.figure(figsize=(8, 5))
for i in range(n_classes):
   plt.plot(fpr[i], tpr[i], label=f' {class_names[i]} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], color='navy', linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic for each class')
plt.legend(loc="lower right")
plt.show()
for i in range(n_classes):
   print(f'AUC for  {class_names[i]}: {roc_auc[i]:.2f}')
```

7.3 Hybrid Model (ConvNextT+EfficientNetB0)

```
from tensorflow.keras.applications import ConvNeXtTiny, EfficientNetB0
from tensorflow.keras.layers import Input, Dense, Dropout, Concatenate, GlobalAveragePooling2D
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.regularizers import l2

# Shared input
input_tensor = Input(shape=(128, 128, 3))

# ConvNeXtTiny base
convnext_base = ConvNeXtTiny(weights='imagenet', include_top=False)
convnext_features = convnext_base(input_tensor)
convnext_out = GlobalAveragePooling2D()(convnext_features)

# EfficientNetB0 base
efficientnet_base = EfficientNetB0(weights='imagenet', include_top=False)
efficientnet_features = efficientnet_base(input_tensor)
efficientnet_out = GlobalAveragePooling2D()(efficientnet_features)

# Combine both features
combined = Concatenate()([convnext_out, efficientnet_out])

# Dense layers
dense1 = Dense(512, activation='relu', kernel_regularizer=l2(0.01))(combined)
dropout1 = Dropout(0.5)(dense1)
output = Dense(6, activation='softmax')(dropout1)

# Define model
hybrid_model = Model(inputs=input_tensor, outputs=output)

# Compile model
hybrid_model.compile(optimizer=Adam(learning_rate=1e-4),
                     loss='categorical_crossentropy',
                     metrics=['accuracy'])
model = hybrid_model
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

x_train = np.reshape(x_train, (x_train.shape[0], x_train.shape[1], x_train.shape[2], 3))

# Create an instance of the ImageDataGenerator with desired augmentation settings
datagen = ImageDataGenerator(

    width_shift_range=0.1,  # Fraction of total width
    height_shift_range=0.1,  # Fraction of total height
    shear_range=0.2,  # Shear intensity
    zoom_range=0.2,  # Zoom range
    horizontal_flip=True,  # Horizontal flipping
    fill_mode='nearest'  # How to fill in newly created pixels
)
```

```
from tensorflow.keras.callbacks import ModelCheckpoint

checkpoint = ModelCheckpoint(
    filepath='hybrid.keras',
    monitor='val_accuracy',
    save_best_only=True,
    save_weights_only=False,
    mode='max',
    verbose=0
)
```

```
history = model.fit(
    datagen.flow(x_train,y_train_encoded,batch_size= 16),
    # x_train,y_train_encoded,batch_size= 32,
    epochs=30,
    validation_data=(x_test, y_test_encoded),
    callbacks=[checkpoint]
)
```

```
import matplotlib.pyplot as plt

# Assuming `history` is the variable returned by model.fit()
plt.figure(figsize=(7, 5))
plt.plot(history.history['accuracy'], label='Train Accuracy' )
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy Curve')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

```
import matplotlib.pyplot as plt

# Assuming `history` is the variable returned by model.fit()
plt.figure(figsize=(7, 5))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss Curve')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

```
from sklearn.metrics import classification_report, accuracy_score, precision_score, recall_score, f1_score
import numpy as np

# Predict the classes
y_pred_probs = model.predict(x_test)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = y_test  # Already in label format

# Calculate metrics
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_true, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_true, y_pred, average='weighted', zero_division=0)

from tensorflow.keras.utils import to_categorical

# One-hot encode y_test for model evaluation
y_test_cat = to_categorical(y_test, num_classes=6)

# Now evaluate
loss = model.evaluate(x_test, y_test_cat, verbose=0)[0]

# Print the results
print(f"Accuracy:  {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall:    {recall:.4f}")
print(f"F1 Score:  {f1:.4f}")
print(f"Loss:      {loss:.4f}")
```

```
from sklearn.metrics import classification_report
import pandas as pd
from tabulate import tabulate
import numpy as np

# Define class names
class_names = ['alive', 'dead', 'downy_mildew', 'growing', 'raising_seeding', 'sclerotinia_rot']

# Predict labels from the model
y_pred_probs = model.predict(x_test)
y_pred_labels = np.argmax(y_pred_probs, axis=1)

# Use y_test directly if it's already in label form
y_true_labels = y_test  # No need to apply argmax

# Generate classification report
report = classification_report(y_true_labels, y_pred_labels, target_names=class_names, output_dict=True)

# Convert to DataFrame
df_report = pd.DataFrame(report).transpose().round(2)

# Display using tabulate
print(tabulate(df_report, headers='keys', tablefmt='fancy_grid'))
```

```
conf_matrix = confusion_matrix(y_test, y_pred_labels)

# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d',
cmap='BuGn',xticklabels=class_names,yticklabels=class_names)
plt.xlabel('Predicted labels')
plt.ylabel('True labels')
plt.title('Confusion Matrix')
plt.show()
```

```
from sklearn.metrics import roc_curve, auc
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import label_binarize
from sklearn.multiclass import OneVsRestClassifier
from sklearn.model_selection import train_test_split
from keras.utils import to_categorical
y_pred = model.predict(x_test)
predictions = np.array(list(map(lambda x: np.round(x), y_pred)))

y_test_bin = label_binarize(y_test, classes=np.unique(y_test))
predictions_bin = label_binarize(predictions, classes=np.unique(y_test))

fpr = dict()
tpr = dict()
roc_auc = dict()

n_classes = len(np.unique(y_test))
for i in range(n_classes):
   fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], predictions_bin[:, i])
   roc_auc[i] = auc(fpr[i], tpr[i])

plt.figure(figsize=(8, 5))
for i in range(n_classes):
   plt.plot(fpr[i], tpr[i], label=f' {class_names[i]} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], color='navy', linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic for each class')
plt.legend(loc="lower right")
plt.show()
for i in range(n_classes):
   print(f'AUC for  {class_names[i]}: {roc_auc[i]:.2f}')
```